3 January 2026

# Beyond-Backpropagation Training: Methods, Applications, and Perspectives

Rongguang Ye, Chenhao Ye, Chao Huang, Ming Tang, Yunhao Liu

## Abstract

Artificial intelligence has revolutionized numerous domains, including computer vision, natural language processing, and autonomous systems. These breakthroughs have been largely driven by the backpropagation (BP) algorithm, which allows neural networks to be trained efficiently. However, BP suffers from limited biological plausibility, computational inefficiency, and memory bottleneck, all of which hinder its applicability to neuromorphic, edge, and large-scale distributed systems. To address these limitations, numerous BP-free algorithms have emerged over the past decade. The rapid growth and variety of these approaches call for a systematic review of existing BP-free methods. In response, this work provides a comprehensive survey of BP-free learning methods, covering representative algorithms across diverse domains. Specifically, we propose a unified taxonomy based on two dimensions-how update signals are computed, and when updates are applied. Building on this taxonomy, we analyze fundamental properties of BP-free methods and compare their performance against BP counterparts, which offers practical guidelines for selecting these methods across different domains. We further highlight key open challenges for future research, including improving theoretical understanding, scaling to deeper networks, and developing biologically inspired learning rules built upon brain-like architectures. We hope this work could provide a roadmap for researchers exploring biologically plausible and resource-efficient training.

## Keywords

biological plausibility, bp-free learning, computing and processing, parallel updates, resource-efficient training

Posted on 3 January 2026 — CC-BY-NC-SA 4.0 — This is a preprint and has not been peer reviewed. Data may be preliminary. — https://doi.org/10.36227/techrxiv.176740426.63642005/v1

# Beyond-Backpropagation Training: Methods, Applications, and Perspectives

RONGGUANG YE*, Southern University of Science and Technology, China
CHENHAO YE*, Southern University of Science and Technology and Nanjing University of Posts and Telecommunications, China
CHAO HUANG, Montclair State University, USA
MING TANG†, Southern University of Science and Technology, China
YUNHAO LIU, Tsinghua University, China

Artificial intelligence has revolutionized numerous domains, including computer vision, natural language processing, and autonomous systems. These breakthroughs have been largely driven by the backpropagation (BP) algorithm, which allows neural networks to be trained efficiently. However, BP suffers from limited biological plausibility, computational inefficiency, and memory bottleneck, all of which hinder its applicability to neuromorphic, edge, and large-scale distributed systems. To address these limitations, numerous BP-free algorithms have emerged over the past decade. The rapid growth and variety of these approaches call for a systematic review of existing BP-free methods. In response, this work provides a comprehensive survey of BP-free learning methods, covering representative algorithms across diverse domains. Specifically, we propose a unified taxonomy based on two dimensions — how update signals are computed, and when updates are applied. Building on this taxonomy, we analyze fundamental properties of BP-free methods and compare their performance against BP counterparts, which offers practical guidelines for selecting these methods across different domains. We further highlight key open challenges for future research, including improving theoretical understanding, scaling to deeper networks, and developing biologically inspired learning rules built upon brain-like architectures. We hope this work could provide a roadmap for researchers exploring biologically plausible and resource-efficient training.

Additional Key Words and Phrases: BP-free learning, biological plausibility, parallel updates, resource-efficient training

## 1 Introduction

The backpropagation (BP) algorithm [1] lies at the core of modern deep learning, enabling effective gradient computation through the chain rule. This mechanism has powered remarkable advances across diverse applications, such as computer vision [2, 3], natural language processing [4, 5], and speech signal processing [6]. Despite its widespread adoption, BP's reliance on global gradient propagation gives rise to several fundamental challenges:

- **Biological implausibility**: BP relies on several assumptions that conflict with biological learning, whose local and energy-efficient nature has inspired alternative training mechanisms. The most critical is the weight transport problem [7]. That is, each layer must use the exact transpose of the downstream weights to compute its gradients, implying that backward connections mirror the forward weights. In real neural circuits, however, neurons cannot access or precisely replicate the weights of upper layers [8]. BP also assumes bidirectional connectivity between layers, which is inconsistent with the brain's asynchronous computations [7, 9]. These discrepancies prevent BP from modeling how real neural circuits learn and limit its potential to inspire energy-efficient and locally trainable learning mechanisms observed in biological systems.

*These authors contributed equally to this research.
†Corresponding author.

Authors' Contact Information: Rongguang Ye, yerg2023@mail.sustech.edu.cn, Southern University of Science and Technology, China; Chenhao Ye, ychbert@outlook.com, Southern University of Science and Technology and Nanjing University of Posts and Telecommunications, China; Chao Huang, huangch@montclair.edu, Montclair State University, USA; Ming Tang, tangm3@sustech.edu.cn, Southern University of Science and Technology, China; Yunhao Liu, yunhao@tsinghua.edu.cn, Tsinghua University, China.

Table 1. A summary of representative surveys. FG: Forward Gradient; ZO: Zeroth-Order Optimization; PG: Proxy Gradient; PC: Predictive Coding; TP: Target Propagation; EP: Equilibrium Propagation; ASS: Activation-Statistics Signal; HB: Hebbian Rule; ER: Evolutionary Rule. Domain Scope indicates the number of domains covered by reviews on BP-free algorithm applications.

| Reference | Year | Update Signals | | | | | | | | | Domain Scope |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | FG | ZO | PG | PC | TP | EP | ASS | HB | ER | |
| [13] | 2020 | ✗ | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | **Single-domain** |
| [14] | 2022 | ✗ | ✗ | ✓ | ✗ | ✓ | ✗ | ✗ | ✗ | ✗ | **Single-domain** |
| [23] | 2023 | ✓ | ✗ | ✓ | ✓ | ✗ | ✓ | ✗ | ✓ | ✗ | **Single-domain** |
| [24] | 2024 | ✓ | ✓ | ✓ | ✗ | ✗ | ✗ | ✓ | ✗ | ✓ | **Single-domain** |
| [20] | 2024 | ✗ | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | **Single-domain** |
| [25] | 2024 | ✗ | ✗ | ✗ | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | **Single-domain** |
| [26] | 2025 | ✗ | ✗ | ✗ | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | **Multi-domain** |
| Ours | 2025 | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | **Multi-domain** |

- **Computational inefficiency**: BP enforces a strict layer-wise dependency between forward and backward computations, leading to *forward and backward locking*. Each layer must wait for the preceding layer to complete during forward propagation, and for the succeeding layer to complete during backward propagation, which severely limits parallelism and training throughput [10, 11].
- **Memory bottleneck**: BP requires retaining intermediate activations during the forward pass, resulting in substantial memory usage [12]. These limitations significantly restrict the applicability of BP to large-scale or resource-constrained systems such as neuromorphic or edge platforms.

These challenges have motivated the exploration of BP-free learning paradigms that can either replace or complement traditional BP. Over the past decade, researchers have explored a diverse set of BP-free learning mechanisms [13, 14]. Early studies were motivated by the pursuit of *biologically plausible learning*—drawing inspiration from synaptic plasticity [15], predictive coding [16], and energy-based models [17]—while more recent work focuses on handling *computational inefficiency* and *memory bottleneck* [18–20]. These approaches rethink the propagation of error signals, parameter update rules, and even network architectures, with the aim of achieving one or more of the following goals: *biological plausibility, computational efficiency, and memory usage efficiency*. The potential of BP-free training is further amplified by the rise of neuromorphic hardware, such as Intel Loihi [21] and IBM TrueNorth [22], which provides event-driven and highly parallel substrates that naturally support these forms of computation. Traditional BP is poorly matched to such systems due to its need for global gradient information and dense memory access, whereas BP-free methods—with their local, lightweight, or parallel update rules—align far more closely with the constraints and opportunities of neuromorphic hardware.

We provide a summary of existing representative surveys on BP-free training in Table 1. While these works provide valuable insights and guidance for researchers, they are often limited in scope. Specifically, some surveys highlight only specific BP-free approaches, such as zeroth-order optimization [20] and predictive coding [25, 26], while others are limited to BP-free techniques in particular application domains, such as signal processing [13] and large language models (LLMs) [20, 24]. To address this gap, we aim to provide a comprehensive overview of BP-free training. We review representative methods across main update-signal categories, propose a new taxonomy based on *how* and *when* updates are performed, analyze their advantages and limitations, and provide practical guidelines for selecting BP-free methods for different applications. Our contributions are as follows:

- To the best of our knowledge, this survey provides the most comprehensive overview of modern BP-free training techniques. We systematically summarize a wide range of representative studies across diverse tasks

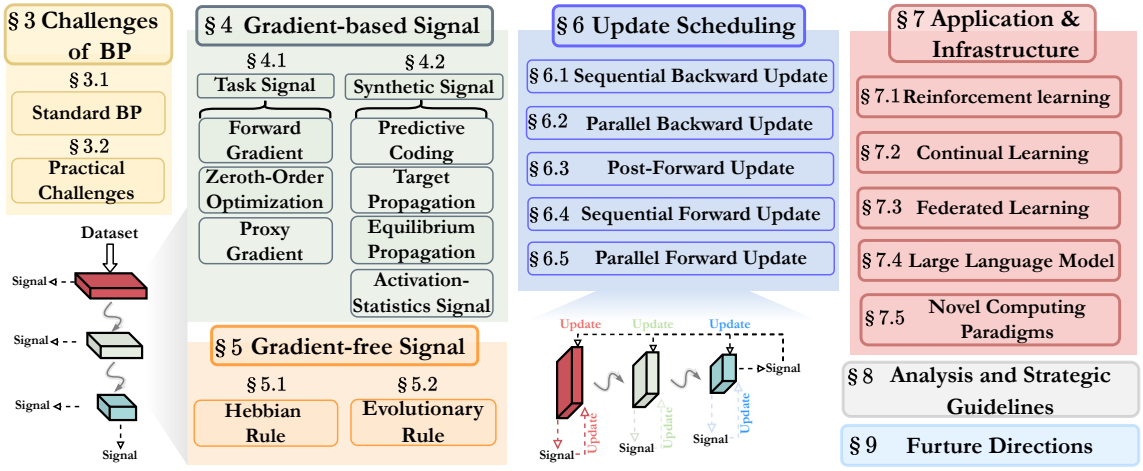



Fig. 1. An overview of the organization in this survey.

and domains, and characterize them along two fundamental dimensions—*how learning signals are generated* and *model parameters are updated*. Based on these dimensions, we establish a unified taxonomy, as shown in Fig. 1, which systematically organizes and covers a broad spectrum of BP-free approaches.

- We conduct a qualitative analysis of BP-free methods, examining their ability to mitigate issues such as forward and update locking, and comparing their learning behaviors and performance gaps relative to standard BP. This analysis provides an understanding of the strengths and limitations of different BP-free approaches. Based on this analysis, we offer actionable guidelines to help researchers and practitioners select appropriate BP-free approaches for specific requirements, including scenarios where computational or memory resources are limited.
- We highlight several promising directions for future research aimed at addressing current limitations of BP-free learning. Key avenues include advancing theoretical understanding to better characterize learning behavior, developing methods that scale efficiently to deeper networks, and tackling the incompatibility between biologically inspired learning principles and artificial neural networks.

## 2 Taxonomy and Organization

To systematically organize and compare BP-free methods, this survey provides the first perspective to categorize them in two dimensions in terms of ① how learning signals are generated for model updapte and ② when parameter updates occur. The first dimension captures the type of learning signal, highlighting whether a method relies on gradient information or alternative rules. This distinction often determines the method's biological plausibility, and its potential for scalability to deeper network architectures. The second dimension captures the timing of parameter updates, which can occur during the forward pass or in a backward-like phase, thereby affecting computational efficiency. Based on these considerations, we propose to organize BP-free methods along these two key dimensions.

① **How learning signals are generated for model update.** Based on this criteria, we categorize BP-free learning signals into two types: *gradient-based signal*, which retains some form of gradient information to guide learning, and *gradient-free signal*, which do not rely on gradients and instead update parameters using alternative rules or heuristics.

- *Gradient-based signal (Sec. 4):* This type of signal avoids full BP by estimating or synthesizing gradients locally instead of propagating exact gradients through the entire computational graph. They can be further divided into two types: **Task signal** (Sec. 4.1) is derived directly from the task objective (e.g., the cross-entropy loss for classification tasks); **Synthetic signal** (Sec. 4.2) generates surrogate gradients through auxiliary modules or learned predictors. Illustrations of different task signals and synthetic signals will be provided in Fig. 3 and Fig. 4, respectively.
- *Gradient-free signal (Sec. 5):* Gradient-free signals discard explicit gradient computation. They update model parameters using alternative rules or search strategies. Such approaches can offer advantages in terms of biological plausibility and robustness to non-differentiable components, making them attractive in settings where gradients are difficult or costly to obtain.

② **When parameter updates occur.** This dimension captures the timing of updates, which defines an orthogonal dimension distinguishing methods by when learning signals are applied. For example, some BP-free methods perform updates during the forward computation for immediate learning, while others defer updates to a backward-like phase to approximate a global signal. Notably, this dimension is directly related to the locking problems, which will be discussed in Problem 2 of Section 3.

Fig. 1 summarizes the overall organization of this survey. Specifically, Section 3 reviews BP and highlights its limitations. Sections 4–5 examine the first dimension of our taxonomy, where Section 4 introduces gradient-based learning signals and Section 5 discusses gradient-free ones. Section 6 examines different update mechanisms, focusing on when model parameters are updated. Section 7 presents the cross-domain applications of current BP-free methods. Section 8 provides a comparative analysis of BP-free methods in terms of computational locking, scalability, and biological plausibility, and offers guidance for researchers from different domains on applying BP-free approaches. Finally, Section 9 suggests future research directions.

## 3 Challenges of Backpropagation

### 3.1 Standard Backpropagation

Without loss of generality, we consider a feedforward neural network with $L$ layers. The weight and bias of layer $l$ are denoted by $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$, respectively. For each layer $l$, let $\mathbf{z}^{(l)} \in \mathbb{R}^{d_l}$ denote the pre-activation of layer $l$ (i.e., the linear input before the activation function), and let $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ denote the post-activation of layer $l$ (i.e., the output after applying the activation function). Fig. 2 provides a schematic illustration of $\mathbf{z}^{(l)}$ and $\mathbf{h}^{(l)}$. The forward pass of layer $l$ is given by

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{h}^{(l)} = \sigma(\mathbf{z}^{(l)}), \tag{1}$$

where $\sigma(\cdot)$ is an element-wise activation function and $\mathbf{h}^{(0)}$ is the network input.

Let $\mathcal{L}$ denote the task-specific loss. According to the chain rule, the error signal at layer $l$ can be expressed as

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \left(\mathbf{W}^{(l+1)}\right)^{\top} \boldsymbol{\delta}^{(l+1)} \odot \frac{\partial \sigma(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(\mathbf{l})}}, \tag{2}$$

where $\odot$ denotes element-wise multiplication. The gradient of the loss with respect to the weight matrix of layer $l$ is computed as

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{h}^{(l-1)})^{\top}. \tag{3}$$

This gradient is then used in a standard gradient descent algorithm to update the weights $\mathbf{W}^{(l)}$, while the bias term is updated analogously via gradient descent on its associated gradient. BP provides exact gradients to allow effective optimization with gradient-based methods. By systematically propagating the error signal from the output layer to earlier layers, it achieves precise error signal propagation and has become the cornerstone of modern deep learning.

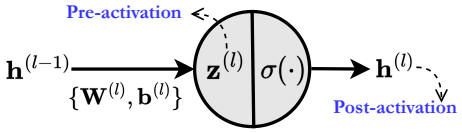



Fig. 2. Illustration of pre-activation and post-activation in a neural network layer.

### 3.2 Practical Challenges

The computational characteristics of BP give rise to several well-known challenges:

Problem 1 (Weight Transport Problem [7]). *The weight transport problem refers to the requirement that each layer must use the exact transpose of the next layer's weight matrix* $(\mathbf{W}^{(l+1)})^\top$ *to compute its local error signal* $\boldsymbol{\delta}^{(l)}$ *in Eq. (2). This dependency on non-local parameters prevents purely local learning and contradicts biological plausibility, where a neuron can only access information locally.*

Problem 2 (Locking Problem [10]). *The locking problem refers to the sequential dependencies inherent in standard BP, which prevent concurrent computation and parameter updates across layers. It manifests in two forms:*

(i) Forward Locking: *The activation of layer* $l$ *cannot be computed until layer* $l-1$ *finishes its forward computation as shown in Eq. (1), which limits forward-pass parallelism.*

(ii) Backward Locking: *The error signal* $\boldsymbol{\delta}^{(l)}$ *depends on* $\boldsymbol{\delta}^{(l+1)}$ *in Eq. (2), preventing parallel backward propagation.*

Problem 3 (Memory Usage Problem [27]). *The memory usage problem occurs because BP must keep all intermediate activations* $\mathbf{h}^{(l)}$ *from the forward pass to compute gradients in the backward pass. In deep networks, this memory cost scales with the number of model parameters, which becomes a major bottleneck for large-scale, heterogeneous, or resource-constrained devices.*

These challenges collectively highlight the limitations of standard BP and motivate the development of alternative training methods. Specifically, the **weight transport problem** stems from the way gradients are computed and propagated across layers, directly motivating approaches that modify *how* learning signals are generated or approximated. In contrast, **locking problem** and **high memory usage**, concern *when* parameter updates occur, since updates must wait for the completion of forward and backward computations or for the availability of intermediate activations. Categorizing BP-free methods along these two dimensions—*how* learning signals are produced and *when* parameters updates occurs–provides a principled framework for analyzing and comparing different approaches.

## 4 Gradient-based Signal

Gradient-based signals are employed by most BP-free approaches. In this section, we focus on how these methods construct alternative learning signals to approximate gradients locally, enabling efficient and modular training. We classify these signals into *task signal*, which is directly derived from the task objective, and *synthetic signal*, which is generated via auxiliary modules or learned predictors.

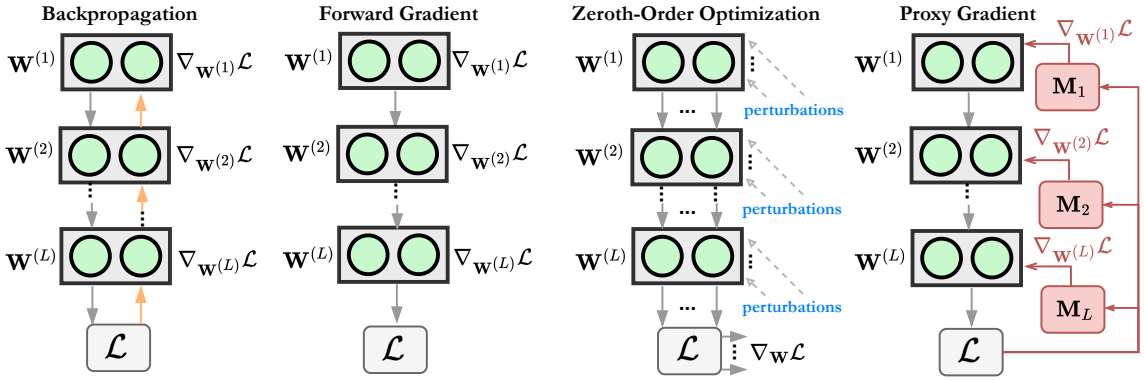



Fig. 3. Illustrations of three task-signal mechanisms compared with standard BP. Green circles represent model weights. $\mathbf{M}_i$ in proxy-gradient methods indicates a surrogate gradient estimator.

### 4.1 Task Signal

As shown in Fig. 3, task signals for BP-free methods can be categorized into three types. *Forward gradient* (Sec. 4.1.1) estimates the gradient along a single direction with only one forward pass, making it computationally efficient but limited to specific directions. *Zeroth-order optimization* (Sec. 4.1.2) approximates gradients by performing multiple forward passes along randomly sampled directions, trading efficiency for more general gradient information. *Proxy gradient* (Sec. 4.1.3) introduces gradient estimators that approximate the global error signal, enabling each layer to perform local updates.

*4.1.1 Forward Gradients.* Forward gradients [28–31] provide a biologically plausible and memory-efficient alternative to traditional BP by estimating directional derivatives of the loss.

Formally, let $\mathcal{L} : \mathbb{R}^m \to \mathbb{R}^n$ denote the loss function. Given a direction vector $\mathbf{v} \in \mathbb{R}^m$, let $J \in \mathbb{R}^{n\times m}$ be the Jacobian of $\mathcal{L}$. Forward gradients estimate the directional derivative of the loss along $\mathbf{v}$, which can be efficiently computed using forward-mode automatic differentiation (AD). Forward-mode AD can be implemented in two ways: ❶ by perturbing the weights directly or ❷ by perturbing the activations of each layer. Both approaches are mathematically equivalent, but they differ in implementation and memory trade-offs. Modern libraries such as `JAX` [32] provide efficient implementations of forward-mode AD.

**❶ Forward-mode weight perturbation [31].** For the weight matrix $\mathbf{W}^{(l)}$ of layer $l$, and a perturbation direction $\mathbf{v}^{(l)}$ of the same shape, the gradient estimator of the directional derivative is

$$J^{(l)}\mathbf{v}^{(l)} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}_{ij}} v^{(l)}_{ij}, \tag{4}$$

where $i$ and $j$ index the rows and columns of the weight matrix $\mathbf{W}^{(l)}$. Intuitively, Eq. (4) measures how much the loss would change if the weights were slightly perturbed along the direction $\mathbf{v}^{(l)}$. Accordingly, it can be used to update the weights as

$$\mathbf{W}^{(l)} \longleftarrow \mathbf{W}^{(l)} - \eta \left(J^{(l)}\mathbf{v}^{(l)}\right)\mathbf{v}^{(l)}, \tag{5}$$

where $\eta$ denotes the learning rate. Note that the directional derivative $J^{(l)}\mathbf{v}^{(l)}$ is a scalar, and in practice it is often absorbed into $\eta$, simplifying the update. Weight perturbation is straightforward to implement. However, it scales poorly

with model size because each perturbation must be applied to all parameters simultaneously, leading to high memory usage in large networks.

To address these limitations, several extensions have been proposed. DODGE [28] reduces the high variance of gradient estimation by generating high-quality candidate directions, where the direction vector is defined as $\mathbf{v}^{(l)} = \frac{\boldsymbol{g}^{(l)}}{||\boldsymbol{g}^{(l)}||}$, and the gradient $\boldsymbol{g}^{(l)}$ can be approximated using deterministic policy gradient or synthetic gradient methods. FoMoH [33] further improves scalability by jointly evaluating directional derivatives and their second-order terms via hyper-dual numbers. WSG-RFG-G [34] adapts forward-mode weight perturbation to spiking neural networks, introducing layer-wise perturbations that enable parallel gradient computation and reduce memory overhead. More recently, Flügel et al. [35] enhanced forward-mode gradient estimation by combining multiple perturbation directions through orthogonal projections, effectively lowering estimation error and improving optimization performance.

❷ **Forward-mode activation perturbation [36].** Equivalently, the directional derivative can be interpreted by perturbing the activations at a given layer. Let $u_j^{(l)}$ be a small perturbation applied to the post-activation $h_j^{(l)}$ of layer $l$, where $j$ indexes the neurons in layer $l$. The gradient estimator of the directional derivative $\mathbf{u}^{(l)}$ is

$$J^{(l)}\mathbf{u}^{(l)} = \sum_j \frac{\partial \mathcal{L}}{\partial h_j^{(l)}} u_j^{(l)}, \tag{6}$$

which quantifies how small changes in the activations affect the loss. The perturbation vector $\mathbf{u}^{(l)}$ specifies a direction in the activation space of layer $l$, and the scalar $J^{(l)}\mathbf{u}^{(l)}$ gives the directional derivative of the loss along this activation perturbation. Because $J^{(l)}\mathbf{u}^{(l)}$ is a scalar, it can be absorbed into the learning rate $\eta$, controlling the step size without affecting the update direction. The weights of layer $l$ can then be updated as

$$\mathbf{W}^{(l)} \longleftarrow \mathbf{W}^{(l)} - \eta\, \mathbf{u}^{(l)} (\mathbf{h}^{(l-1)})^\top, \tag{7}$$

where multiplying $\mathbf{u}^{(l)}$ by $(\mathbf{h}^{(l-1)})^\top$ propagates the directional information $\mathbf{u}^{(l)}$ from activations to weights. Compared to weight perturbation, activation perturbation typically requires less memory and enables local layer-wise updates, making it particularly appealing for hardware-efficient implementations.

Forward-mode activation perturbation estimates gradients along a single forward direction. While conceptually simple, this approach suffers from high variance and poor scalability, especially for large networks. To mitigate this issue, LG-FG [37] computes forward gradients using local contrastive losses, thereby reducing the dimensionality of each gradient estimation. Other solutions focus on identifying more effective local perturbation directions, for example by leveraging a small auxiliary model [38] or neural tangent kernel approximations [39] to guess the direction vector, thereby reducing variance and improving the reliability of gradient approximation. Recently, Daniel et al. [40] enhanced forward-mode activation perturbation by reusing and manipulating layer-wise Jacobian information propagated from earlier layers to form more accurate guess directions, effectively reducing both bias and variance while improving scalability to larger networks.

*4.1.2 Zeroth-Order (ZO) Optimization.* ZO methods approximate gradients by observing the effect of parameter perturbations on the loss, relying exclusively on function evaluations rather than internal model computations. This property makes ZO particularly well-suited for black-box optimization scenarios [41–44].

ZO methods can be broadly categorized two types based on the form of the directional vector used for gradient estimation: ❶ coordinate-wise gradient estimation and ❷ randomized vector-wise gradient estimation.

❶ **Randomized vector-wise gradient estimation (RGE) [43].** For each layer $l$, RGE estimates $\nabla_{\mathbf{W}^{(l)}}\mathcal{L}(\mathbf{W})$ by applying random perturbations to the parameters $\mathbf{W}^{(l)}$. The estimated gradient is obtained as the average over $n$ random perturbations:

$$\bar{\nabla}_{\mathbf{W}^{(l)}}\mathcal{L}(\mathbf{W}^{(l)}) = \frac{1}{n}\sum_{i=1}^{n}\left[\frac{\mathcal{L}\left(\mathbf{W}^{(l)} + \mu\boldsymbol{v}_{(i)}^{(l)}\right) - \mathcal{L}(\mathbf{W}^{(l)})}{\mu}\boldsymbol{v}_{(i)}^{(l)}\right], \tag{8}$$

where $\boldsymbol{v}_{(i)}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ denotes a random perturbation sampled from a Gaussian distribution, $n$ is the number of sampled directions, and $\mu > 0$ is a small perturbation step size. The parameters of each layer $l$ can then be updated as

$$\mathbf{W}^{(l)} \longleftarrow \mathbf{W}^{(l)} - \eta\,\bar{\nabla}_{\mathbf{W}^{(l)}}\mathcal{L}(\mathbf{W}^{(l)}). \tag{9}$$

We state a proposition characterizing a key property of RGE:

Proposition 1 (Unbiasedness of RGE [44]). *RGE provides an* unbiased *stochastic estimate of the true gradient. Formally, for a weight matrix $\mathbf{W}^{(l)}$ at layer $l$, let $\boldsymbol{v}_{(i)}^{(l)}$ denote the $i$-th independent random perturbation drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Then, the expectation over these perturbations satisfies $\mathbb{E}_{\boldsymbol{v}_{(i)}^{(l)}}\left[\boldsymbol{v}_{(i)}^{(l)}\left((\boldsymbol{v}_{(i)}^{(l)})^{\top}\nabla_{\mathbf{W}^{(l)}}\mathcal{L}(\mathbf{W}^{(l)})\right)\right] = \nabla_{\mathbf{W}^{(l)}}\mathcal{L}(\mathbf{W}^{(l)})$.*

Although unbiased, RGE typically suffers from high variance, and its estimation quality depends on the trade-off between the number of sampled perturbations and the dimensionality of the parameter space. In particular, the number of sampled directions $n$ plays a crucial role in controlling the estimator variance. Empirical analyses, such as those in MeZO [45], show that using a single perturbation ($n = 1$) leads to extremely high variance. More generally, theoretical results [44, 46] indicate that the variance of the RGE estimator scales as $O(|\mathbf{W}|/n)$.

❷ **Coordinate-wise gradient estimation (CGE) [13].** Unlike RGE, CGE adopts deterministic perturbations along each coordinate direction of the weights $\mathbf{W}^{(l)}$:

$$\bar{\nabla}_{\mathbf{W}^{(l)}}\mathcal{L}(\mathbf{W}^{(l)}) = \frac{1}{m}\sum_{i=1}^{m}\left[\frac{\mathcal{L}(\mathbf{W}^{(l)} + \mu\boldsymbol{e}_{(i)}^{(l)}) - \mathcal{L}(\mathbf{W}^{(l)})}{\mu}\boldsymbol{e}_{(i)}^{(l)}\right], \tag{10}$$

where $\boldsymbol{e}_{(i)}^{(l)}$ denotes the $i$-th standard basis vector, and $m$ is the number of sampled coordinates in layer $l$. The parameters of each layer are also updated based on Eq. (9).

However, the CGE estimation requires one forward evaluation per dimension, making it computationally prohibitive for high-dimensional models. Empirical studies show that when the number of sampled coordinates $m$ matches the number of model parameters, CGE can achieve higher estimation accuracy than RGE [47]. To improve efficiency, DeepZero [47] applies ZO perturbations to sparsified models, effectively reducing the dimensionality of the perturbation space and reusing intermediate features for parallel computation, thereby enhancing scalability to large models.

Since both RGE and CGE suffer from a trade-off between estimation accuracy and computational cost, subsequent studies have been developed to address these challenges. Momentum-based ZO approaches incorporate historical gradient information to stabilize estimation [48, 49], while structure-based methods exploit gradient sparsity or low-rank priors to reduce perturbation dimensionality [50–52]. In addition, several works [45, 53, 54] employ seed mapping to reconstruct gradient directions on the fly, drastically lowering memory usage without sacrificing estimation quality.

*4.1.3 Proxy Gradient.* Proxy-gradient methods update each layer using locally generated or approximated error signal $\boldsymbol{\delta}_l$, avoiding the need for a full backward pass. Formally, the error signal of layer $l$ can be approximated as

$$\boldsymbol{\delta}^{(l)} \approx \mathbf{M}_l(\mathcal{I}_l), \tag{11}$$

where $\mathbf{M}_l(\cdot)$ denotes a learned or random local mapping, and $\mathcal{I}_l$ represents the information accessible at layer $l$. Different proxy-gradient methods mainly differ in how $\mathbf{M}_l$ and $\mathcal{I}_l$ are instantiated. Notably, once the local error signal $\boldsymbol{\delta}^{(l)}$ is obtained from Eq. (11), the gradient for layer $l$ can be computed as $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)}(\mathbf{h}^{(l-1)})^\top$, which is then used to update the weights $\mathbf{W}^{(l)}$ of layer $l$.

In the following, we present representative proxy-gradient methods—including the ❶ feedback alignment family [55, 56], ❷ decoupled neural interfaces [57], and ❸ NoProp [58].

**❶ Feedback alignment family.** To recall, the weight transposition problem (Problem 1) arises because BP requires the transpose of the $(l+1)$-th layer weights, $(\mathbf{W}^{(l+1)})^\top$, when computing the error signal for layer $l$ (see Eq. (2)). Feedback Alignment (FA) [55] addresses this issue by replacing $(\mathbf{W}^{(l+1)})^\top$ with a random matrix $\mathbf{B}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} = \begin{cases} \mathbf{B}^{(l)}\boldsymbol{\delta}^{(l+1)}, & l < L, \\ \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}}, & l = L, \end{cases} \tag{12}$$

where $\mathbf{B}^{(l)}$ is initialized randomly (e.g., from a Gaussian distribution) at the start of training and then kept fixed throughout training. Although $\mathbf{B}^{(l)}$ is not equal to the true transpose $(\mathbf{W}^{(l+1)})^\top$, using fixed random feedback matrices still allows training to proceed. This is because the forward weights $\mathbf{W}^{(l)}$ gradually adapt during learning, effectively rotating such that the fixed random matrix $\mathbf{B}^{(l)}$ transmits signals increasingly aligned with the true gradient. This emergent alignment phenomenon is known as feedback alignment. The success of FA has inspired several extensions and variants. For instance, RFLO [59] adapts FA to recurrent neural networks by applying random FA through time. KP-WM [60] employs a weight-mirror mechanism that makes $\mathbf{B}^{(l)}$ learnable, ensuring that it remains proportional to $\mathbf{W}^{(l)}$ rather than identical to it, thereby avoiding the weight-transport problem.

Nevertheless, in FA, the layer-wise error propagation relies on successive transformations through intermediate layers, which makes the error signal propagation inefficient—especially in deep networks where lower layers receive weak or noisy feedback. Direct feedback alignment (DFA) [56] addresses this limitation by allowing each layer to receive error signals directly from the output layer, bypassing the need for sequential BP through intermediate layers. Specifically, each layer $l$ uses a random matrix $\mathbf{B}^{(l)}$ to project the output error $\boldsymbol{\delta}^{(L)}$ back to that layer:

$$\boldsymbol{\delta}^{(l)} = \begin{cases} \mathbf{B}^{(l)}\boldsymbol{\delta}^{(L)}, & l < L, \\ \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}}, & l = L. \end{cases} \tag{13}$$

This direct feedback pathway allows all layers to receive the final-layer error signal simultaneously, enabling parallel updates and faster convergence in deep networks.

Indirect feedback alignment (IFA) [56] further extends FA by addressing a practical limitation of both FA and DFA: the need to store and maintain a separate random feedback matrix for every layer. Such layer-wise random matrices incur non-negligible memory overhead, especially for deep or wide networks. IFA alleviates this issue by introducing an indirect feedback pathway, where learning signals for all layers are derived from a single random projection. Unlike FA and DFA, IFA performs layer updates in a forward order—from the first layer to the last layer. Specifically, in IFA, the output error is first projected to the first hidden layer using a single fixed random matrix $\mathbf{B}^{(1)}$:

$$\boldsymbol{\delta}^{(1)} = \mathbf{B}^{(1)}\boldsymbol{\delta}^{(L)}. \tag{14}$$

After computing the error for the first layer, the parameters of this layer can be updated locally. The error is then propagated forward to subsequent layers for sequential updates:

$$\boldsymbol{\delta}^{(l)} = \mathbf{W}^{(l)}\boldsymbol{\delta}^{(l-1)}, \quad l = 2, ..., L. \tag{15}$$

❷ **Decoupled Neural Interfaces (DNI).** DNI leverage proxy-gradient methods to predict local error signals, enabling fully decoupled layer updates. Each layer $l$ is equipped with a learnable module $\mathbf{M}_l$ (e.g., a shallow multilayer perceptron), which estimates the gradient of the loss with respect to its activations:

$$\hat{\boldsymbol{\delta}}^{(l)} = \mathbf{M}_l(\mathbf{h}^{(l)}), \quad l = 1, ..., L-1, \tag{16}$$

where $\hat{\boldsymbol{\delta}}^{(l)}$ approximates the true error signal $\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$. At the output layer, the error signal is given directly by $\boldsymbol{\delta}^{(L)}$. During the forward pass, the weights $\mathbf{W}^{(l)}$ are immediately updated based on

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l} \approx \hat{\boldsymbol{\delta}}^{(l)}(\mathbf{h}^{(l-1)})^\top. \tag{17}$$

To learn model $\mathbf{M}_l$, DNI introduces a recursive error signal derived from the chain rule, where for each layer the true error signal can be expressed as

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l+1)}} \approx \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{z}^{(l)}} \hat{\boldsymbol{\delta}}^{(l+1)}, \quad l = 1, ..., L-1. \tag{18}$$

This equation indicates that each layer's true error signal $\boldsymbol{\delta}^{(l)}$ can be obtained recursively. Each $\mathbf{M}_l$ is then trained by minimizing the squared difference between its prediction and the true error signal $\boldsymbol{\delta}^{(l)}$, i.e., $||\hat{\boldsymbol{\delta}}^{(l)} - \boldsymbol{\delta}^{(l)}||^2$.

❸ **NoProp.** Most recently, NoProp presents a radical departure from prior proxy-gradient or feedback-alignment methods. Rather than computing gradients or passing activations layer-to-layer, each layer is cast as a denoising module: given a noisy target embedding (derived from the true label), the layer learns to map toward a clean target embedding. Formally, let $\mathbf{e}_\mathbf{y}$ denote a learned embedding for class label $\mathbf{y}$, and let $\tilde{\mathbf{e}}_{\mathbf{y},l}$ be a noisy version for layer $l$, obtained by adding Gaussian perturbation to $\mathbf{e}_\mathbf{y}$. Each layer's forward computation $f_l$, parameterized by $\mathbf{W}^{(l)}$, is optimized by

$$\min_{\mathbf{W}^{(l)}} \left\| f_l(\tilde{\mathbf{e}}_{\mathbf{y},l}; \mathbf{W}^{(l)}) - \mathbf{e}_\mathbf{y} \right\|^2. \tag{19}$$

This optimization objective trains each layer to map its noisy input $\tilde{\mathbf{e}}_{\mathbf{y},l}$ back toward the clean target $\mathbf{e}_y$ without receiving global gradient signals. At inference time, the sequence of layers jointly perform progressive denoising of the target embedding, culminating in the classification decision.

Similar to NoProp's local learning approach, PredSim [61] and SID [11] compute the cross-entropy loss locally using the labels at each layer, allowing layer-wise updates to be performed in parallel without relying on full BP.

### 4.2 Synthetic Signal

Synthetic signals refer to artificially designed training signals or loss functions that are not directly derived from the task objective. Fig. 4 illustrates four types of synthetic signals. In *Predictive Coding*, after the forward computation at each layer $l$, the weights $\mathbf{W}^{(l)}$ are reused to reconstruct the activations of the lower layer. *Target propagation* also reconstructs layer activations while introducing a separate model $\mathbf{V}^{(l)}$ for reconstruction, with label-driven supervision applied at the output layer. *Equilibrium propagation* treats activations and weights as two coupled states and updates them by minimizing a defined energy function. *Activation-statistics signals* leverage statistics over positive and negative samples to maximize activations for positive inputs while minimizing them for negative ones.

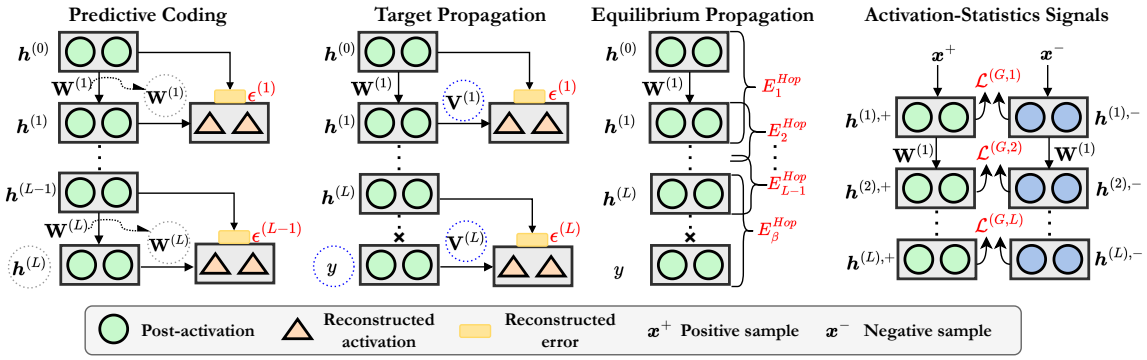



Fig. 4. Illustration of the learning mechanisms for the four types of synthetic signal. Red-font symbols indicate the update signals.

*4.2.1 Predictive Coding (PC).* Instead of simply propagating signals forward, PC operates by having each higher layer predict the activity of the layer below, and the resulting reconstruction error—the difference between prediction and actual activity—guides the learning.

In the following, we describe how PC can be applied to both ❶ unsupervised and ❷ supervised learning scenarios.

**❶ PC for unsupervised learning.** In the activation reconstruction formulation, the activation of layer $(l-1)$ given the upper layer $l$ is modeled as a Gaussian distribution:

$$p(\mathbf{h}^{(l-1)} \mid \mathbf{h}^{(l)}) = \mathcal{N}(\hat{\mathbf{h}}^{(l-1)}, \Sigma^{(l-1)}), \tag{20}$$

where the predicted mean of the lower layer $\hat{\mathbf{h}}^{(l-1)}$ is computed using the same weights $\mathbf{W}^{(l)}$ and bias $\mathbf{b}^{(l)}$ as the forward computation $\hat{\mathbf{h}}^{(l-1)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)})$, and $\Sigma^{(l-1)}$ denotes the prediction covariance. PC constructs the self-supervised learning (SSL) signal that encourages each layer to form representations that more accurately predict the activation of the layer below, thereby improving the quality of layer-wise representations relative to a purely feedforward pass. The SSL objective is to maximize the negative log-likelihood of observed activations across all layers:

$$E = \sum_{l=1}^{L} \boldsymbol{\epsilon}^{(l)} = \sum_{l=1}^{L} -\log p(\mathbf{h}^{(l-1)} \mid \mathbf{h}^{(l)}). \tag{21}$$

Training proceeds in two alternating stages (i.e., neural stage update and weight update). In the neural stage update, neuronal activations $\mathbf{h}^{(l)}$ are iteratively updated to minimize $E$ while keeping weights fixed:

$$\mathbf{h}^{(l)} \leftarrow \mathbf{h}^{(l)} - \eta \frac{\partial E}{\partial \mathbf{h}^{(l)}}, \tag{22}$$

where $\eta$ is a small step size. In the subsequent *weight update stage*, weights are updated locally at each layer $l$:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} + \eta \left( \boldsymbol{\epsilon}^{(l-1)} \odot \frac{\partial \sigma\left(\mathbf{z}^{(l)}\right)}{\partial \mathbf{z}^{(l)}} \right) \left(\mathbf{h}^{(l)}\right)^{\top}. \tag{23}$$

These two phases are iteratively alternated until convergence, allowing each layer to capture the structure of the layer below and form self-consistent predictive representations without any external labels.

**❷ PC for supervised learning.** In supervised learning tasks, PC allows error signals from the output target to drive the entire network [62, 63]. In this setting, supervised energy function combines all layer-wise prediction errors with a

supervised signal:

$$E = \frac{\lambda}{2} \sum_{l=1}^{L-1} \|\boldsymbol{\epsilon}^{(l)}\|^2 + \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{h}}^{(L)}\|^2, \tag{24}$$

where $\mathbf{y}$ is the target label, $\lambda$ controls the strength of intermediate layer consistency, and $\boldsymbol{\epsilon}^{(l)}$ is defined as in Eq. (21). This energy function balances two contributions: the first term enforces consistency between successive layers by minimizing the layer-wise prediction errors $\boldsymbol{\epsilon}^{(l)}$, effectively acting as a regularizer that aligns hidden representations; the second term drives the network toward the supervised target $\mathbf{y}$, ensuring correct output predictions. As in the unsupervised case, the activations and weights are iteratively updated using Eq. (22) and Eq. (23), respectively.

It has been shown that, in the supervised PC setting multi-layer perceptrons (MLPs), once the neural states reach equilibrium (i.e., $\frac{\partial E}{\partial \mathbf{h}^{(l)}} \to 0$), the local prediction errors $\boldsymbol{\epsilon}^{(l)}$ are proportional to the gradients of the global BP gradient [62], i.e., $\boldsymbol{\epsilon}^{(l)} \propto \frac{\partial \mathcal{L}_{\text{BP}}}{\partial \mathbf{z}^{(l)}}$, where $\frac{\partial \mathcal{L}_{\text{BP}}}{\partial \mathbf{z}^{(l)}}$ denotes the gradient of standard BP. This reveals that the supervised PC, despite relying solely on local computations, can approximate standard BP in multilayer networks. Extending this idea, Millidge et al. [63] demonstrated that the supervised PC can approximate BP not only in MLPs, but also along arbitrary computation graphs, including convolutional, recurrent, and LSTM networks.

However, the two-stage updates of activations and weights make supervised PC inefficient. To adress this, Z-IL [64] replaces the iterative optimization of PC with a single-pass computation of local error signals. By recursively propagating errors through local derivatives, each layer obtains a gradient-like signal directly. The Z-IL update rule is mathematically equivalent to backpropagation and yields identical gradients.

Recent works have addressed the instability and poor performance of PC networks in deep architectures, enabling them to perform well in deeper networks. For example, $\mu$PC [65] introduces the Depth-$\mu$P technique [66] to stabilize forward propagation and prevent exploding or vanishing activations in PC networks. Qi et al. [67] enables stable training of deeper PC networks by balancing error propagation and adjusting weight updates, achieving performance comparable to BP. Meanwhile, DPC [68] was proposed as a novel weight update mechanism that reduces error accumulation in deeper layers. To further enhance flexibility, $PC_{\mathcal{F}_{KL}}$ [69] removes the Gaussian assumption of traditional PC. Instead, it approximates non-Gaussian energy functions using Monte Carlo samples, enabling the framework to handle more general probabilistic objectives. DCPC [70] further generalizes PC to structured target distributions. By representing the target distribution with populations of Monte Carlo samples, DCPC decomposes the problem into local coordinate updates of individual random variables, going beyond the Gaussian assumption.

*4.2.2 Target Propagation (TP).* TP [71] is a supervised learning method that follows the same principle as PC in using reconstruction-based self-supervised signals. However, unlike supervised PC, TP employs separate inverse parameters for reconstruction instead of reusing the forward weights and biases. Given a feedforward network with activations $\mathcal{H} = \{\mathbf{h}^{(1)}, \ldots, \mathbf{h}^{(L)}\}$, forward parameters $\mathbf{W} = \{\mathbf{W}^{(1)}, \ldots, \mathbf{W}^{(L)}\}$, and inverse parameters $\mathbf{V} = \{\mathbf{V}^{(1)}, \ldots, \mathbf{V}^{(L)}\}$, each inverse mapping $g^{(l)}(\cdot\,; \mathbf{V}^{(l)})$ (typically implemented as a neural network layer) is trained to approximate the local inverse of $f^{(l)}(\cdot\,; \mathbf{W}^l)$:

$$g^{(l)}(f^{(l)}(\mathbf{h}^{(l-1)}\,; \mathbf{W}^{(l)}); \mathbf{V}^{(l)}) \approx \mathbf{h}^{(l-1)}, \tag{25}$$

where $f^{(l)}$ is the forward computation at layer $l$. To improve robustness and prevent trivial identity mappings, one can add noise to the activations. In this case, the input to $g^{(l)}$ in Eq. (25) changes from the output of the forward computation, $f^{(l)}(\mathbf{h}^{(l-1)}; \mathbf{W}^{(l)})$, to the noisy activation of the current layer, $\tilde{\mathbf{h}}^{(l)}$, and the inverse mapping now directly

predicts the noisy activation of the previous layer, $\tilde{\mathbf{h}}^{(l-1)}$:

$$\min_{\mathbf{V}^{(l)}} \left\| g^{(l)}(\tilde{\mathbf{h}}^{(l)}; \mathbf{V}^{(l)}) - \tilde{\mathbf{h}}^{(l-1)} \right\|^2, \tag{26}$$

where $\tilde{\mathbf{h}}^{(l)} = \mathbf{h}^{(l)} + \xi$ with $\xi \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ denotes the noisy activation of layer $l$. After training the inverse mappings $g^{(l)}$, each layer computes a local prediction $\hat{\mathbf{h}}^{(l)} = g^{(l)}(\tilde{\mathbf{h}}^{(l+1)}; \mathbf{V}^{(l)})$ and updates its forward weights by

$$\mathbf{W}^{(l)} \longleftarrow \mathbf{W}^{(l)} - \eta \frac{\partial}{\partial \mathbf{W}^{(l)}} \left\| f^{(l)}(\mathbf{h}^{(l-1)}; \mathbf{W}^{(l)}) - \hat{\mathbf{h}}^{(l)} \right\|^2. \tag{27}$$

Each layer thus learns from a local target without propagating gradients through the full network.

Early TP formulations [71] relied on learned inverse mappings to propagate targets from the output layer backward. However, in deep networks, the inversion performed by $g^{(l)}$ is often inaccurate. To address this, difference target propagation (DTP) [72] introduced a correction term to stabilize deep training:

$$\tilde{\mathbf{h}}^{(l-1)} = \underbrace{g^{(l)}(\tilde{\mathbf{h}}^{(l)}; \mathbf{V}^{(l)})}_{\text{TP target}} + \underbrace{\left(\mathbf{h}^{(l-1)} - g^{(l)}(\mathbf{h}^{(l)}; \mathbf{V}^{(l)})\right)}_{\text{correction term}}. \tag{28}$$

The correction term compensates for the reconstruction error of the inverse model and prevents the accumulation of inversion errors in deep networks.

Subsequent research has extended TP in multiple directions. Recursive and local update schemes [73] generalized TP to continuous and recurrent networks, allowing hidden-layer targets to be refined iteratively through local computations rather than relying on full backward passes. At the architectural level, block-wise and greedy training strategies, such as decoupled greedy learning [74], leveraged TP principles to independently train network modules, enabling parallelization and improving efficiency. Extensions to neuromorphic and spiking networks adapted local target updates [75] to continuous-time synaptic dynamics, demonstrating TP's applicability beyond MLPs.

*4.2.3 Equilibrium Propagation (EP).* EP [76] is a supervised learning algorithm inspired by contrastive learning in continuous Hopfield networks [77]. Building on this perspective, EP formulates parameter learning as the result of local equilibrium dynamics in an energy-based system. Recall that $\mathcal{B} = \{\mathbf{b}^{(1)}, \ldots, \mathbf{b}^{(L)}\}$ denote the biases for each layer, and $\mathcal{Z} = \{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(L)}\}$ is the pre-activations for each layer. The Hopfield-type energy function in EP is defined as

$$E^{Hop}(\mathcal{H}, \mathcal{Z}, \mathcal{B}, \mathbf{W}) = \sum_{l=1}^{L} \left( -\frac{1}{2} \sum_{i,j} \mathbf{W}_{ij}^{(l)} \mathbf{h}_i^{(l)} \mathbf{h}_j^{(l)} - \sum_i \mathbf{b}_i^{(l)} \mathbf{h}_i^{(l)} + \frac{1}{2} \sum_i \left(\mathbf{z}_i^{(l)}\right)^2 \right), \tag{29}$$

where the first term captures the pairwise interactions between neurons weighted by $\mathbf{W}^{(l)}$, promoting activation patterns that align with the learned connectivity; the second term incorporates the layer biases $\mathbf{b}^{(l)}$, guiding neurons toward preferred activation levels; and the third term penalizes large pre-activations $\mathbf{z}^{(l)}$, ensuring that the network dynamics converge to a stable equilibrium. Typically, EP consists of the following three phases.

❶ **Free phase.** In this phase, neural activations are iteratively updated to reach a stable equilibrium that minimizes the network's energy. Specifically, the network relaxes its hidden activations $\mathcal{H}$ to an equilibrium state $\mathcal{H}^0$ that minimizes

$$\mathcal{H}^0 = \arg\min_{\mathcal{H}} E^{Hop}(\mathcal{H}, \mathcal{Z}, \mathcal{B}, \mathbf{W}). \tag{30}$$

During this phase, only the activations $\mathcal{H}$ are updated, while weights and biases remain fixed.

❷ **Nudged phase.** During the *nudged phase*, a weak supervised signal is applied to the output layer, and activations are updated to a new equilibrium reflecting this perturbation. Formally, the energy in the nudged phase is defined as

$$E_\beta^{Hop}(\mathcal{H}, \mathcal{Z}, \mathcal{B}, \mathbf{W}) \;=\; E^{Hop}(\mathcal{H}, \mathbf{W}) + \beta\, \mathcal{L}(\mathbf{h}^{(L)}, \mathbf{y}), \tag{31}$$

where $\mathcal{L}(\mathbf{h}^{(L)}, \mathbf{y})$ is a loss value (e.g., squared error or cross-entropy), and $\beta > 0$ is a small scalar. The network then relaxes to the perturbed equilibrium

$$\mathcal{H}^\beta \;=\; \arg\min_{\mathcal{H}} \; E_\beta^{Hop}(\mathcal{H}, \mathcal{Z}, \mathcal{B}, \mathbf{W}), \tag{32}$$

producing activations slightly biased toward the target $\mathbf{y}$.

❸ **Weight update phase.** Following the two equilibrium phases, weight update is performed by contrasting the energy gradients of the free and nudged phases. By differentiating the perturbed equilibrium condition, the weight update can be expressed as

$$\mathbf{W}^{(l)} \longleftarrow \mathbf{W}^{(l)} + \frac{1}{\beta}\Big(\mathcal{H}^{\beta,(l)}(\mathcal{H}^{\beta,(l-1)})^\top - \mathcal{H}^{0,(l)}(\mathcal{H}^{0,(l-1)})^\top\Big), \tag{33}$$

where $\mathcal{H}^{0,(l)}$ and $\mathcal{H}^{\beta,(l)}$ denote the activations of layer $l$ in the free and nudged phases, respectively. In theory, the equivalence between this weight update and the gradient of the loss holds in the limit $\beta \to 0$; in practice, a small finite $\beta$ is used to approximate the gradient.

Building upon the original EP framework, several limitations have been addressed in subsequent work. A major challenge is the stability and convergence of deeper or more complex networks due to the iterative relaxation of activations; this has been mitigated by reformulating EP within a bilevel optimization framework using implicit differentiation, providing a principled link to modern optimization theory [78]. The implementation of EP in analog or neuromorphic hardware presents difficulties because continuous equilibrium dynamics are hard to realize; this limitation is overcome by end-to-end training of analog neural circuits with equilibrium dynamics [79]. Generalization of equilibrium learning beyond classical networks requires new formulations; extending EP to quantum systems [80] allows gradient-based learning to naturally emerge from quantum dynamics. The gap between theoretical EP and practical hardware deployment poses further challenges; hybrid gradient computation schemes [81] for digitally-tied analog blocks offer a solution by combining digital and analog updates efficiently.

*4.2.4 Activation-Statistics Signal (ASS).* ASS encompasses learning algorithms in which each layer updates its weights based on local statistics of the activations, without relying on BP.

The forward-forward (FF) algorithm [82] is the foundational approach. For each input, the network performs two forward passes: one with positive (real) data $\mathbf{x}^+$ and another with negative (corrupted or generated) data $\mathbf{x}^-$. Each layer computes a local goodness measure $G^{(l)}(\mathbf{h}^{(l)})$, for example, the sum of squared activations. The layer-wise learning objective can be expressed as

$$\mathcal{L}^{(G,l)} = -\Big(G^{(l)}(\mathbf{h}^{(l),+}) - G^{(l)}(\mathbf{h}^{(l),-})\Big), \tag{34}$$

and the weights $\mathbf{W}^{(l)}$ of layer $l$ are updated to minimize this objective. This formulation preserves the essence of FF: each layer improves its goodness for positive inputs and suppresses it for negative inputs.

However, the original FF cannot directly make use of labels. Predictive forward-forward (PFF) extends the original FF by incorporating predictive coding-inspired mechanisms while retaining the positive-negative goodness evaluation. Each layer consists of two interacting modules: the representation circuit (RC), which is responsible for optimizing the weights; the generative circuit (GC), which performs a target-propagation-like process using self-supervised

reconstruction signals as local objectives, while in the final layer it denoises the representations using task-specific supervised signals to ultimately recover the target labels. RC computes the standard goodness measure $G^{(l)}(\mathbf{h}^{(l)})$ for positive and negative samples, exactly as in FF (i.e., Eq. (34)). The RC module update weights $\mathbf{W}^{(l)}$ as follows:

$$\mathbf{W}^{(l)} \longleftarrow \mathbf{W}^{(l)} - \eta \left( \frac{\partial \mathcal{L}^{(G,l)}}{\partial \sum_j^{d_l} (\mathbf{h}_j^{(l)})^2} \odot \frac{\partial \sigma(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \right) (\mathbf{h}^{(l-1)})^\top, \tag{35}$$

where $d_l$ denotes the number of neurons in layer $l$.

The GC, following the target propagation principle with parameter $\{\mathbf{V}^{(1)}, ..., \mathbf{V}^{(L)}\}$, maintains predicted activations $\hat{\mathbf{h}}^{(l)}$ and computes a local reconstruction error $\boldsymbol{\epsilon}^{(l)} = \mathbf{h}^{(l)} - \hat{\mathbf{h}}^{(l)}$, where $\hat{\mathbf{h}}^{(l)}$ is consistent with the reconstruction estimate in TP (i.e., Eq. (25)). This local reconstruction error is used to define a quadratic loss $\mathcal{L}^{(GC,l)} = \frac{1}{2}\|\boldsymbol{\epsilon}^{(l)}\|^2$, and performing a single-step gradient descent on this loss yields the following update rule:

$$\mathbf{V}^{(l)} \longleftarrow \mathbf{V}^{(l)} - \eta \boldsymbol{\epsilon}^{(l)} (\mathbf{h}^{(l+1)})^\top. \tag{36}$$

This directly moves $\mathbf{V}^{(l)}$ to lower the layer $l$ reconstruction error. The output layer is subsequently updated by

$$\tilde{\mathbf{h}}^{(L)} \longleftarrow \tilde{\mathbf{h}}^{(L)} - \eta \frac{\partial ||\tilde{\mathbf{h}}^{(L)} - \mathbf{y}||^2}{\partial \tilde{\mathbf{h}}^{(L)}}. \tag{37}$$

This separation allows the RC to perform forward-only error propagation based on positive and negative examples, while the GC refines layer representations using reconstruction signals. Together, they enable each layer to improve both discriminative and reconstruction properties.

Subsequent works have extended the FF paradigm by addressing its limitations in exploiting activation statistics for layer-wise learning. ReduNet [83, 84] observes that naive FF objectives may not fully leverage the class structure in high-dimensional activations. To address this, it optimizes a layer-wise rate-reduction objective, promoting inter-class separability and intra-class compactness, and constructs a white-box network where each layer is explicitly optimized for richer local representations. Mono-Forward [85] identifies that FF still relies on relatively simple positive/negative passes. It introduces a forward-only algorithm that computes layer-specific targets and updates weights based solely on local signals, improving efficiency and parallelism. ForwardGNN [86] extends FF to graph-structured data. It updates node embeddings using local forward signals derived from graph topology and higher-layer information.

## 5 Gradient-free Signal

In this section, we discuss two types of gradient-free signals: Hebbian rules (Sec. 5.1) and evolutionary rules (Sec. 5.2).

### 5.1 Hebbian Rule

Hebbian learning (HL) is a fundamental mechanism in neuroscience, often summarized as "neurons that fire together, wire together" [15, 87–89]. It describes how the connection strength between neurons changes in response to correlated pre- and post-activations. Mathematically, for the weight $\mathbf{W}_{ij}^{(l)}$ connecting neuron $i$ to neuron $j$, the simplest update is

$$\mathbf{W}_{ij}^{(l)} \longleftarrow \mathbf{W}_{ij}^{(l)} + \eta\, \mathbf{h}_i^{(l-1)}\, \mathbf{h}_j^{(l)}, \tag{38}$$

where $\mathbf{h}_i^{(l-1)}$ and $\mathbf{h}_j^{(l)}$ denote the pre- and post-activations, and $\eta$ is a learning rate. This rule implements the core Hebbian principle: the weight grows whenever the two related activations are simultaneously high.

While this simple rule captures the basic correlation-based idea, it cannot make use of target information and therefore does not directly support supervised learning. Early extensions address this limitation by contrasting network activations under different conditions. A representative example is contrastive HL [90], which introduces two phases (i.e., free and clamped phase) that encode the network's behavior before and after imposing supervision.

In the free phase, the network processes the input $\mathbf{x}$ in its natural forward pass, producing a baseline set of activations $\{\mathbf{h}^{(1)-}, ..., \mathbf{h}^{(L)-}\}$. In the clamped phase, the output layer is forced toward the target $\mathbf{y}$, and the activations of the preceding layers are adjusted to minimize the network's error under output layer's constraint. Formally, the clamped activations $\{\mathbf{h}^{(1)+}, \ldots, \mathbf{h}^{(L-1)+}\}$ are obtained by solving

$$\{\mathbf{h}^{(1)+}, \ldots, \mathbf{h}^{(L-1)+}\} = \arg \min_{\mathbf{h}^{(1)}, ..., \mathbf{h}^{(L-1)}} \mathcal{L}\big(\mathbf{h}^{(1)}, \ldots, \mathbf{h}^{(L-1)}, \mathbf{h}^{(L)} = \mathbf{y}\big), \tag{39}$$

where $\mathcal{L}$ is a task-related loss (e.g., cross-entropy for classification).

The weight update is computed from the difference between the two correlation terms:

$$\mathbf{W}_{ij}^{(l)} \longleftarrow \mathbf{W}_{ij}^{(l)} + \eta\Big(\mathbf{h}_i^{(l-1)+}\mathbf{h}_j^{(l)+} - \mathbf{h}_i^{(l-1)-}\mathbf{h}_j^{(l)-}\Big). \tag{40}$$

Intuitively, the weight update encourages the network to adopt activation patterns that yield the correct output while suppressing those that occur naturally without supervision. It is like reinforcing the pattern that matches the target while canceling the network's unconstrained tendency.

SOFTHEBB [91] further extends HL to soft winner-take-all networks with a probabilistic interpretation. The weight update from input neuron $i$ to output neuron $j$ is

$$\mathbf{W}_{ij}^{(l)} \longleftarrow \mathbf{W}_{ij}^{(l)} + \eta\, \mathbf{h}_j^{(l)} \left(\mathbf{h}_i^{(l-1)} - \rho_j \mathbf{W}_{ij}^{(l)}\right), \quad \rho_j = \sum_i \mathbf{W}_{ij}^{(l)} \mathbf{h}_i^{(l-1)}. \tag{41}$$

Here, $\mathbf{h}_j^{(l)}$ ensures that weight changes are proportional to post-activation, consistent with HL. The term $(\mathbf{h}_i^{(l-1)} - \rho_j \mathbf{W}_{ij}^{(l)})$ implements a soft competition: each neuron distributes its total activation across its inputs, so that connections corresponding to the strongest inputs are selectively strengthened while weaker or redundant connections are reduced. This encourages neurons to form sparse, specialized receptive fields that focus on the most salient features of the input.

While SOFTHEBB focuses on improving local competition among inputs, other approaches aim to extend Hebbian updates to account for global error information. One such method is PEPITA [92], which performs a second forward pass with the input slightly modulated to reflect the network's output error. This modulation embeds a global error signal into the neurons' activations, allowing each neuron to adjust its weights based on how it contributes to the overall error. In this way, Hebbian updates remain local, but they are guided by a global learning signal.

While PEPITA extends Hebbian updates to incorporate global error signals, another line of research focuses on making HL more biologically plausible in hierarchical networks. BDSP [93] addresses the observation that neurons often communicate via bursts of spikes, which are more effective in driving synaptic (i.e., weight) changes than isolated spikes. The weight connecting neuron $i$ in layer $l$ to neuron $j$ in layer $l+1$ is updated as

$$\mathbf{W}_{ij}^{(l+1)} \longleftarrow \mathbf{W}_{ij}^{(l+1)} + \eta\, \mathbf{c}_j^{(l+1)}\, \mathcal{T}(\mathbf{h}_i^{(l)}, \mathbf{z}_j^{(l+1)}), \quad l = 0, ..., L-1, \tag{42}$$

where $\mathbf{c}_j^{(l+1)}$ indicates whether neuron $i$ in layer $l+1$ is active, and $\mathbf{z}_j^{(l+1)}$ is the pre-activation in layer $l+1$. A common way to determine whether neuron $j$ in layer $l+1$ is strongly activated is

$$\mathbf{c}_j^{(l+1)} = \begin{cases} 1, & \mathbf{h}_j^{(l+1)} > \theta_{\text{burst}}, \\ 0, & \text{otherwise}, \end{cases} \tag{43}$$

where $\theta_{\text{burst}}$ is a threshold. Then, the Hebbian-like function $\mathcal{T}$ can be implemented as

$$\mathcal{T}(\mathbf{h}_i^{(l)}, \mathbf{z}_j^{(l+1)}) = \mathbf{h}_i^{(l)} \cdot (\mathbf{z}_j^{(l+1)} - \theta_{\text{dend}}), \tag{44}$$

where $\theta_{\text{dend}}$ is a dendritic threshold that determines when the pre-activation of neuron $j$ in layer $l+1$ is strong enough to trigger Hebbian-like weight changes, and $i$ indexes a neuron in layer $l$. This formulation preserves the locality of Hebbian updates while allowing neuron activations and pre-activation thresholds to modulate the weight change. Learning emerges from the local interaction of input activations, output gating, and pre-activation levels, without requiring explicit backpropagation. Here, $\mathbf{h}_i^{(l)}$ drives the weight increase, $\mathbf{c}_j^{(l+1)}$ gates updates for strongly active output neurons, and $\mathbf{z}_j^{(l+1)}$ determines whether the activation is above a threshold for potentiation or depression.

### 5.2 Evolutionary Rule

Inspired by biological evolution, researchers have turned to evolutionary learning rules, which view network training not as a single gradient trajectory but as a process of population-level search.

Early work on evolutionary artificial neural networks (EANNs) [94] laid the foundation for this idea, proposing to optimize both network weights and topologies through mechanisms akin to mutation, recombination, and selection. Rather than updating a single parameter vector via gradient descent, evolution-based methods evolve a population of networks, each evaluated by a fitness (e.g., loss or reward value) function. Over generations, high-performing individuals guide the distribution toward promising regions of the search space.

A central formulation of this paradigm is the evolution strategy (ES), which approximates the gradient of expected fitness through stochastic sampling:

$$\mathbf{W} \longleftarrow \mathbf{W} + \frac{\eta}{N\sigma} \sum_{k=1}^{N} R(\mathbf{W} + \epsilon \boldsymbol{v}_k)\, \boldsymbol{v}_k, \tag{45}$$

where $R(\cdot)$ is the fitness function (e.g., accuracy for classification), $\boldsymbol{v}_k \sim \mathcal{N}(0, I)$ are perturbations, $\epsilon$ controls search variance, $\sigma > 0$ denote a normalized factor, and $N$ is the number of individuals.

Although early comparisons suggested that evolution strategies were less efficient than backpropagation [95], their robustness and parallelism became key advantages as networks grew larger. Salimans et al. [96] demonstrated that ES could scale to deep reinforcement learning tasks with tens of millions of parameters, offering remarkable stability and distributed efficiency. This insight reignited interest in evolutionary training of deep and even language models [97], showing that large-scale neuroevolution could rival gradient-based methods in performance.

Beyond classical ES, many variants explored different inspirations from nature. For example, Particle Swarm Optimization [98] replaces the typical random perturbation and selection of candidate networks with a mechanism inspired by collective behavior: each particle (representing a network) updates its parameters by combining information from its own best performance so far and the best performance observed across the population. Simulated annealing [99] introduces temperature-controlled randomness to escape local minima, allowing sparse or modular networks to emerge. Structural evolution strategies [100] further extend this idea to jointly evolve weights and architectures, anticipating

the rise of neural architecture search. Recent efforts have also explored coevolutionary systems [101], where multiple network network sets evolve cooperatively. This mirrors how biological systems evolve specialized but interdependent components, collectively improving the performance of the whole network.

## 6 Update Scheduling

While previous sections categorized methods based on the generation mechanism of learning signals, an equally critical dimension is the timing of parameter updates. We categorize BP-free methods into five regimes based on update timing. *Sequential backward* and *parallel backward* methods update parameters based on feedback from higher layers, with the difference being whether the feedback is propagated recursively or broadcasted. *Post-forward* methods defer updates until the network has completed a full forward pass, eliminating the traditional backward structure. *Sequential forward* and *parallel forward* methods progressively relax inter-layer dependencies, allowing layers to update asynchronously.

### 6.1 Sequential Backward Update

Methods in this category maintain the topological causality of the backward pass found in BP: the error signal for layer $l$ is explicitly derived from the error signal of its successor layer $l+1$. As a result, these methods are still subject to forward and backward locking, requiring sequential computations and updates. However, by replacing the exact chain rule with alternative mechanisms, they can improve biological plausibility or reduce the memory burden.

**Random Feedback Alignment.** These methods approximate BP by replacing the weight transpose in the backward pass with fixed random matrices, thereby avoiding the biologically implausible weight transport. For example, in FA [55], the error signal for layer $l$, denoted as $\boldsymbol{\delta}^{(l)}$, is computed as $\boldsymbol{\delta}^{(l)} = \mathbf{B}^{(l)}\boldsymbol{\delta}^{(l+1)}$. Because the computation of $\boldsymbol{\delta}^{(l)}$ strictly depends on $\boldsymbol{\delta}^{(l+1)}$, parameter updates must proceed sequentially from the output layer back to the input layer.

**Recursive Local Error.** These strategies compute local error signals for each layer in a manner that each layer's error depends on signals from other layers. Unlike standard BP, which requires storing all activations and performing sequential backward passes, these methods calculate local errors directly in a single forward-like pass. Nevertheless, because each layer's error depends on signals from other layers, the parameter updates retain a sequential backward dependency. A representative example is the Z-IL framework [64], which computes local errors in a single forward-like pass while producing exact gradients equivalent to standard BP, reducing memory requirements and improving computational efficiency.

### 6.2 Parallel Backward Update

Sequential backward in standard BP creates a bottleneck, as each layer must wait for error signals from its successor before updating. Parallel backward update strategies relax this constraint by broadcasting a global signal—such as the output error or target—allowing all layers to compute their updates simultaneously. This eliminates backward locking while still relying on information from the network output.

**Direct Error Broadcasting.** This strategy eliminates the sequential dependency of the backward pass by transmitting a global output error to all hidden layers through fixed projection matrices or simplified signals. Once the forward pass is complete, every layer can compute its parameter updates in parallel, effectively removing backward locking. Typical instantiations of this idea include methods that broadcast full error vectors, like DFA [56], or simplified scalar signals combined with local eligibility traces, as in Kickback [102]. By decoupling layers during the feedback phase, Direct error broadcasting exemplifies a general mechanism for enabling parallel backward updates.

**Direct Target Projection.** Target signals (e.g., labels) are broadcast directly to hidden layers, enabling parallel backward updates by removing the dependency on the error signals of subsequent layers. Once the forward pass is complete, each layer independently computes its updates using information from both the labels and the output-layer error signals, enabling full parallelization across layers or distributed nodes. Methods like DRTP [103] exemplify this approach, showing that decoupling layer-to-layer dependencies in the feedback phase can simplify error propagation while preserving learning effectiveness.

### 6.3 Post-Forward Update

Post-forward update does not rely on the backward pass, effectively removing backward locking. However, weight updates remain forward-locked, because they depend on at least one full forward computation.

**Global Error Re-injection.** This strategy computes the global error after a standard forward pass and feed it back to modulate the input or network activity. Once this modulated signal propagates through the network, layer-wise updates can be applied. Approaches like PEPITA [92] exemplify this mechanism, where a second forward pass uses the error-modulated input, and weights are updated based on the difference in layer activations between the two passes. The dependence on the global error inherently enforces a forward lock.

**Iterative Relaxation.** Energy-based methods, such as EP [76] and PC [63], update weights only after the network dynamics have settled into an equilibrium state. Typically, the network undergoes a "free phase" with clamped input, followed by a "nudged phase" with the output weakly clamped. The weight update is computed based on the differences between these two equilibrium states, relying on the completion of the full relaxation process rather than an explicit backward pass.

**Perturbation-Based Estimation.** Perturbation-based methods avoid explicit gradient computation by assessing how changes to parameters affect a global scalar objective. Zeroth-order approaches, such as MeZO [45] and BAFFLE [104], perturb weights and measure the resulting change in the loss. Similarly, population-based methods like evolutionary strategies [96] generate a set of perturbed networks and update parameters based on aggregate fitness. In all cases, parameter updates are performed only after completing one or more full forward evaluations, which are used to estimate gradients or gradient-like signals for updating the parameters.

### 6.4 Sequential Forward Update

Sequential forward update methods implement “on-the-fly” learning: each layer updates its parameter immediately during or after processing its specific input, without waiting for the network to produce a global error signal. These methods remove backward locking and allow pipelined computation across layers.

**Activation-Driven Local Adaptation.** Parameter updates are guided by the changes in each layer’s activations, relying solely on local information. For example, in SoftHebb [105], updates depend only on the pre-activation $\mathbf{h}^{(l-1)}$ and post-activation $\mathbf{h}^{(l)}$, without reference to downstream layers. Similarly, greedy feedforward variants [82] optimize a local “goodness” objective: once $\mathbf{h}^{(l)}$ is computed, parameters’ update is applied before the signal propagates to layer $l+1$, making the update timing sequential and forward-oriented.

**Target-Augmented Local Learning.** In this strategy, updates are derived from the forward-propagated supervision signal at each layer. For instance, SigProp [106] augments the forward activation with the target vector. As layer $l$ processes the input, it simultaneously transforms the target signal. A local loss is then computed between the activation and the transported target, enabling immediate updates.

**Forward-Mode Differentiation.** Forward-mode methods [31] compute directional derivatives of the loss with respect to parameters alongside the activations using dual numbers. As the forward pass propagates through the network, the gradient trace for each layer accumulates without storing activations for a separate backward phase. While the final scalar error is required to complete the gradient computation, the structural accumulation proceeds sequentially along the forward pass, enabling immediate preparation of layer-wise updates.

**Local Reconstruction.** These methods use layer-wise reconstruction signals or prediction errors as local supervisory cues to guide weight updates. In target propagation and its variants, such as DTP [107], each layer computes a local target or reconstruction signal using a newly introduced inverse-mapping model of the next layer. The layer then computes a reconstruction error based on the difference between its actual activation and the reconstructed target, and updates its weights accordingly. In contrast, predictive coding frameworks [25, 63] compute layer-wise prediction errors by reusing the current forward-pass weights during activation reconstruction, reducing memory burden without requiring an explicit inverse model. In both cases, updates are driven by local reconstruction signals, enabling layers to adjust weights sequentially during the forward pass.

### 6.5 Parallel Forward Update

The most decoupled regime is parallel forward update, where layers operate asynchronously. By breaking the strict dependency on the immediate predecessor's output, these methods eliminate both forward and backward locking, enabling massive parallelization.

**Synthetic Interfaces.** This approach decouples a layer from its predecessor by providing a synthetic input. For instance, in DNI [57], an auxiliary network predicts the input and the local error signal for layer $l$, allowing it to compute activations and update weights without waiting for the computations of other layers. By generating locally usable signals, synthetic interfaces enable asynchronous layer-wise updates and facilitate distributed or pipelined training.

**Replay Buffers.** Instead of synthesizing inputs, this strategy relies on memory mechanisms to decouple adjacent layers temporally. In approaches such as asynchronous greedy learning [38], a local replay buffer $\mathcal{B}_{l-1}$ is inserted between layer $l-1$ and layer $l$. Layer $l-1$ pushes its computed representations into the buffer, while layer $l$ samples a batch from the buffer to perform its local training. This breaks the tight temporal coupling of the forward pass; layer $l$ can train on data stored from previous timestamps while layer $l-1$ processes new data, effectively enabling pipelined or fully asynchronous execution.

**Independent Sub-Problems.** This strategy decomposes global network training into multiple local tasks, relaxing strict input-output dependencies between layers so that each layer can be trained independently and in parallel. One implementation employs auxiliary variables. For instance, alternating minimization [108] and ADMM [109] introduce layer-wise target variables and soften the hard equality constraints between layers (i.e., $\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)})$) into penalty terms. This decouples the optimization of weights and activations, allowing each layer to solve a local convex problem simultaneously. Another approach, termed target-based independence, treats each layer as a standalone module that computes its activations based on a local denoised target derived from the global denoised objective, as in NoProp [58]. By using these local objectives rather than relying strictly on the outputs of the preceding layer, sequential dependencies are removed, enabling fully parallel layer-wise updates.

## 7 Application and Infrastructure

The principles of BP-free learning have evolved into a versatile toolkit for modern machine learning. These methods offer unique advantages, including biological plausibility, update locality, and low memory footprint. This section

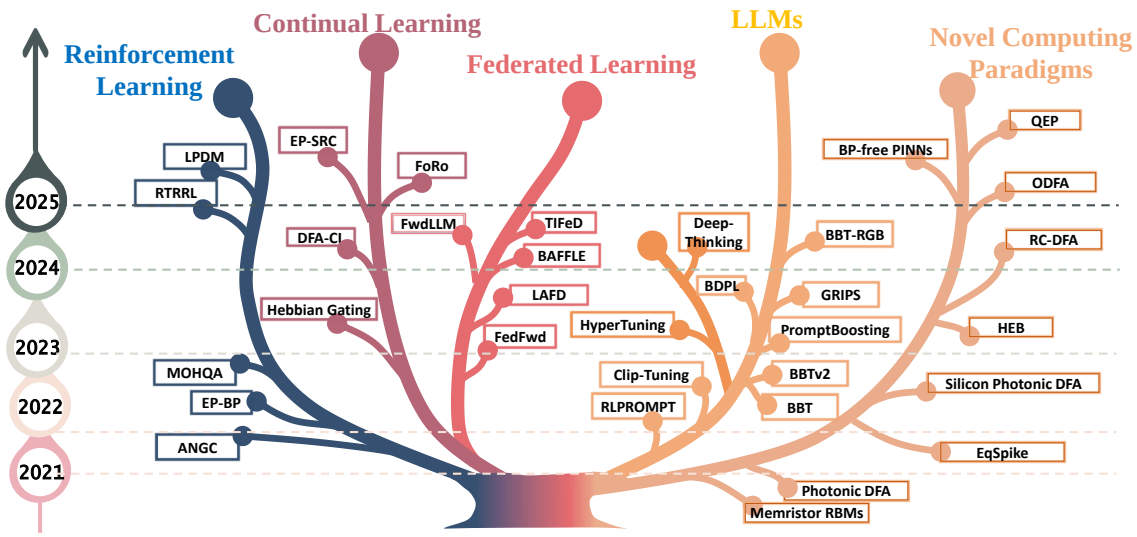



Fig. 5. The evolution and diversification of BP-free learning.

surveys the expansion of BP-free paradigms into five key areas: reinforcement learning, continual learning, federated learning, large language models, and novel computing paradigms.

### 7.1 Reinforcement Learning

In Reinforcement learning (RL), standard BP-based approaches often struggle with the high variance of gradient estimates, the computational cost of propagating errors through long time horizons, and the vanishing gradients. BP-free methods address these limitations through three distinct strategies:

**Hybrid Training.** Standard actor-critic methods train both the actor and critic networks using BP, which can lead to high-variance updates and unstable policy learning. Hybrid approaches address this issue by incorporating energy-based dynamics into the actor update. EP-BP [110] was introduced, in which the critic is trained with BP while the actor is updated using EP [76]. Unlike standard policy gradient methods that minimize instantaneous errors, EP updates follow the manifold of the system's equilibrium states. This approach effectively regularizes the actor update, preventing the "policy collapse" often observed in high-variance BP training, even when the critic itself is trained conventionally.

**Real-Time Online Gradient Approximation.** In partially observable environments, agents must maintain memory of past events. Standard BP through time [111] requires storing the entire history of activations, creating a linear memory bottleneck. RTRRL [112] overcomes this by combining temporal difference learning with biologically plausible rules. By utilizing local eligibility traces to approximate gradients online, this approach eliminates the need to store history, significantly reducing memory overhead and enabling real-time adaptation in complex environments.

**Local Temporal Error Propagation.** BP-free mechanisms can bridge time lags without global error BP to address the "vanishing gradient" problem in long-horizon tasks. MOHQA [113] introduces a parallel Hebbian module alongside a standard Deep Q-Network. It uses "rarely correlated eligibility traces" to rapidly associate past states with delayed

rewards. This Hebbian pathway continues to operate reliably even when the global gradient signal—after many time steps—has become so noisy and diluted that BP can no longer extract meaningful updates.

### 7.2 Continual Learning

Continual learning aims to mitigate catastrophic forgetting, where learning new tasks overwrites previously acquired representations. BP-free methods offer intrinsic solutions to the catastrophic forgetting problem through implicit regularization, dynamic consolidation, and localized plasticity mechanisms.

**Task-Specific Feedback Alignment.** BP-free algorithms can naturally reduce task interference by shaping how feedback signals steer the updates. Folchini et al. [114] showed that using task-specific and mutually orthogonal feedback matrices directs each task's updates along distinct directions. By keeping the updates for different tasks in separate subspaces of the weight matrix, previously learned tasks are less likely to be overwritten, thereby mitigating catastrophic forgetting.

To emulate the brain's ability to switch contexts, Flesch et al. [115] proposed combining "sluggish" task units with Hebbian learning. The local Hebbian updates automatically drive the weights associated with different contexts to become orthogonal. This effectively creates soft task-specific sub-networks within the same architecture, preventing new memories from interfering with old ones without requiring explicit replay buffers.

**Hebbian Replay Consolidation.** Inspired by biological sleep, SRC [116] method combines EP [76] with an unsupervised replay phase. During "sleep", the network generates its own inputs based on learned correlations, and weights are updated via local Hebbian rules. This offline consolidation disentangles task representations in the latent space, achieving state-of-the-art performance for biologically plausible continual learning.

**Goodness-Driven Spiking Plasticity.** Spiking neural networks (SNNs) offer two features beneficial for continual learning: (i) activations are sparse and triggered by discrete events, and (ii) learning rules rely only on temporally local information. Ghader et al. [117] leverage these properties by porting the forward–forward framework [82] to SNNs. Their method uses two forward passes (positive and negative) to compute layer-wise goodness scores and then applies local plasticity rules that depend only on the layer's spike activity. These design choices reduce catastrophic forgetting for several reasons. First, sparse spike activity produces fewer simultaneous synaptic updates, lowering the chance that updates for a new task strongly modify weights learned from previous tasks. Second, local learning driven by layer-specific goodness prevents a single global error from inducing large, coordinated parameter changes (the "ripple effect") across layers. Third, the temporal dimension of spikes lets the network separate task-relevant events in time, which further reduces destructive interference. These factors make layer-wise spiking plasticity a natural mechanism for preserving low-level feature extractors while still permitting effective adaptation to new tasks.

### 7.3 Federated Learning

Federated learning (FL) deploys models across decentralized devices, facing constraints in resources, bandwidth, and privacy. BP-free methods address these bottlenecks by removing the backward pass and enabling non-gradient communication.

**Integer DFA for On-Device Training.** On tiny micro-controllers, the floating-point operations and memory footprint required by BP make on-device training impractical. BP-free methods alleviate these constraints by eliminating the backward graph and enabling update rules that depend only on locally available signals. Building on this property, TIFeD [118] implements DFA [56] entirely with integer arithmetic. Since DFA does not store intermediate activations for a backward pass and uses fixed random feedback matrices, its updates can be executed with minimal state and

without Floating-Point computation. This design allows full on-device training within tight RAM budgets, extending federated learning to the TinyML range of hardware.

**Layer-wise Local Updates.** BP requires storing activations for the entire network, imposing a high memory burden that limits on-device training. BP-free methods such as the forward-forward algorithm circumvent this by allowing layers to optimize local objectives independently during the forward pass. FedFwd [119] leverages this property to perform greedy layer-wise updates: each layer updates its weights immediately as its forward computation completes, and activations are discarded right afterward. This approach drastically reduces the peak memory footprint, enabling deep model training on memory-constrained client devices.

**Privacy-Preserving Local Error Feedback.** In federated learning, transmitting full gradients can expose sensitive client information, and on constrained or inference-only hardware, direct gradient computation may be infeasible. BP-free methods mitigate both issues by avoiding explicit backward passes. The LAFD framework [120] applies BP-free updates using DFA [56]. Each client maintains its own random feedback matrix, allowing weight updates to be computed locally from projected error signals without exposing full gradients. This mechanism inherently preserves client privacy while supporting asynchronous updates across devices.

### 7.4 Large Language Models

Fine-tuning LLMs is challenging when models are accessed as "black boxes" (API-only) or when the optimization objective is non-differentiable. BP-free methods provide the primary means for interaction in these regimes.

**BP-Free Black-Box Optimization.** When gradients are unavailable—such as when accessing closed APIs like GPT-4—standard BP cannot be used to tune prompts. BP-free methods provide alternative optimization strategies that rely entirely on forward evaluations and external optimization rules, making them compatible with frozen LLMs. These methods can be applied to both continuous soft prompts and discrete text tokens.

Continuous prompts are optimized without gradient information using derivative-free strategies. BBT [121] employs evolutionary strategies to optimize prompts projected into a lower-dimensional subspace. Its successor, BBTv2 [122], improves efficiency for deep prompts through a divide-and-conquer approach. Clip-Tuning [123] introduces stochastic dropout to create a mixture of experts from a frozen model, providing richer feedback signals for the evolutionary optimizer. Building on this paradigm, BBT-RGB [124] enhances generalization by decomposing the optimization into orthogonal perspectives: instruction initialization, hierarchical optimization, and verbalizer engineering.

Discrete prompts are inherently non-differentiable, making standard BP inapplicable. BP-free methods leverage evolutionary strategies to optimize tokens directly. EvoPrompt [125] integrates genetic algorithms with LLMs, using the model itself to perform crossover and mutation operations on discrete prompts. Similarly, OPRO [126] treats the LLM as an optimizer by describing the optimization trajectory in natural language, instructing the model to iteratively generate improved prompts based on past performance.

**Zeroth-Order Fine-Tuning.** BP-free methods allow fine-tuning large models under strict memory constraints by eliminating the need to cache activations for BP. For example, MeZO [45] applies zeroth-order optimization to fine-tune full-parameter LLMs using only the memory required for inference MeZO, while ZO-AdaMU [53] incorporates momentum to stabilize the inherently high-variance zeroth-order updates. To further reduce variance in billion-parameter models, SubZero [127] restricts parameter perturbations to a random low-rank subspace, leveraging the observation that effective update directions often lie in a lower-dimensional manifold.

Beyond perturbative approaches, forward-only rules provide an alternative for memory-efficient adaptation. Mem-PEPITA [128] extends the classic PEPITA [92] algorithm to transformer architectures, using a second forward pass

with modulated inputs instead of BP. This enables models such as DistilBERT and GPT-3-small to be trained on memory-constrained devices where BP is infeasible [129]. Similarly, the FoRo framework [130] adapts pre-trained models in a gradient-free manner: task-specific prompts are optimized via evolutionary strategies, and the classifier is updated with recursive least squares, eliminating the memory overhead associated with standard BP.

### 7.5 Novel Computing Paradigms

Conventional BP requires precise arithmetic, global synchronization, and high-precision storage of intermediate activations—constraints that are fundamentally incompatible with emerging non-von Neumann hardware. BP-free learning leverages local computations and the native dynamics of physical substrates, enabling efficient and scalable learning on these novel platforms.

**Neuromorphic Computing.** SNNs process information via asynchronous, discrete events, making the global synchronized chain rule of BP both computationally expensive and physically unnatural. BP-free approaches exploit the temporal and local structure of spikes to perform learning. For example, EqSpike adapts EP [76] to the spiking domain, allowing layer-wise event-driven updates that naturally follow spike timing [131–133]. Similarly, Burst-dependent plasticity rules [93] coordinate hierarchical learning through multiplexed neural signals without requiring a backward pass, aligning with the local processing constraints of neuromorphic chips.

**Photonic Computing.** Optical processors excel at massively parallel matrix multiplications, but cannot implement the sequential nonlinear operations required for the BP backward pass. BP-free methods are well suited for these substrates. Building on this, photonic co-processors have been developed for DFA [134], and scalable optical training of Transformers has been demonstrated via ODFA [135]. In addition, forward-gradient methods allow gradients to be estimated directly on hardware, mitigating calibration errors and further enabling "in-situ" training [136]. To eliminate the need for detailed knowledge of the physical system, model-free, black-box, or gradient-free algorithms have also been proposed, including zeroth-order methods [137, 138] and population-based approaches such as evolutionary strategies and reinforcement learning [139]. Another class of approaches relies on contrastive or energy-based learning, such as EP [76, 140], which updates weights by comparing two equilibrium states under different inputs; experimental implementations include memristor arrays and elastic networks [141]. Finally, Hamiltonian echo BP (HEB) [142] directly produces correct weight updates in time-reversal-invariant Hamiltonian systems without explicit feedback, using physical dynamics and a time-reversal operation, as demonstrated in nonlinear optics experiments.

**Quantum Computing.** BP is generally intractable in parameterized quantum circuits because measuring a quantum state to compute gradients collapses the state, preventing storage of intermediate activations. BP-free approaches mitigate this by exploiting the system's native dynamics. For instance, QEP [143] generalizes classical EP using Onsager reciprocity, allowing all parameter gradients to be extracted from a single perturbation experiment via linear response. This provides a scalable alternative to standard parameter-shift methods, which, in contrast, would require an exponential number of measurements to obtain the same gradients.

## 8 Analysis and Strategic Guidelines

### 8.1 Analysis

To provide a structured comparison of the diverse BP-free landscape, we present a comprehensive qualitative assessment in Tables 2 and 3. Specifically, we analyze mainstream BP-free algorithms and their limitations from three key perspectives: biological plausibility, computational locking, and memory efficiency. Table 2 focuses on fundamental

algorithms—pioneering methods that introduce novel learning mechanisms or distinct update rules. Table 3 lists domain-specific applications where those fundamental principles are adapted to solve specialized problems.

To enable rigorous comparison, we define qualitative criteria for *biological plausibility (Bio.)* and *memory efficiency (Mem.)*, respectively. Biological plausibility is measured by similarity to synaptic plasticity mechanisms, with three levels. **High** (●): biologically inspired update rules (e.g., neuronal activity); **Medium** (◐): methods addressing structural constraints while still using global error signals; **Low** (○): functional optimization approaches with no direct link to neural dynamics. *Memory efficiency* is assessed relative to standard BP: **High** (●) indicates significant memory savings (e.g., no activation caching), **Medium** (◐) is comparable to BP, and **Low** (○) reflects higher memory cost than BP.

The main observations from Tables 2 and 3 are summarized as follows:

**Biological Plausibility.** Table 2 illustrates that the biological plausibility of BP-free algorithms strongly depends on their signal type and gradient dependency. Methods based on local, activity-driven [82], or Hebbian signals [105] achieve high biological plausibility, as they rely solely on pre- and post-synaptic activity without global error propagation. Importantly, high biological plausibility does not necessarily reduce performance: layer-specific feedback approaches such as FF [82], PFF [144], and SoftHebb [105] reach accuracy comparable to gradient-based methods. From application perspective, biological plausibility is particularly crucial for neuromorphic computing, under which biologically-inspired update rules enable efficient computation (Table 3).

**Computational Locking.** Unlocking computational dependencies is a key advantage of certain BP-free algorithms. Methods such as NoProp [58] and PC [16] enable layer-wise or parallel updates, bypassing sequential gradient propagation. These methods maintain performance comparable to standard BP, highlighting their promise for BP-level performance. This capability is particularly valuable for scenarios that would otherwise be prohibitive, such as federated learning on edge devices and fine-tuning large language models (Table 3).

**Memory Efficiency.** As shown in Table 2, methods such as FF [82], PFF [144], PredSim [61], and SID [11] achieve high memory efficiency by using local learning signals to update each layer immediately, thereby avoiding activation storage. Notably, these methods maintain performance comparable to—or even exceeding—that of BP, indicating that well-designed local updates can be both memory-efficient and effective. These properties make them particularly attractive for memory-constrained platforms, including neuromorphic and quantum hardware (Table 3).

### 8.2 Strategic Guidelines for BP-Free Deployment

Based on a comprehensive review of the current landscape of BP-free learning, we provide strategic guidelines tailored to specific application constraints and research objectives.

**Fine-tuning Large Pre-trained Models on Resource-Constrained Devices.** When the primary goal is to adapt large powerful pre-trained models (PTMs) like LLMs on devices with extremely limited memory (e.g., mobile phones, embedded systems), ZO optimization methods stand out as the most pragmatic and effective choice. Methods like MeZO [45] and its enhanced version ZO-AdaMU [53] have demonstrated a remarkable ability to fine-tune models with billions of parameters using memory footprints comparable to inference, a feat unattainable with BP-based methods. This "space-for-time" trade-off is particularly advantageous in scenarios where memory is the hard constraint and a moderate increase in the number of forward passes (computation time) is acceptable. The success of these methods heavily relies on the smooth and well-structured optimization landscape created by pre-training. Therefore, for practitioners, the recommended pipeline is: start with the strongest available PTM, select a parameter-efficient fine-tuning (PEFT) strategy like LoRA [159] or adapters to drastically reduce the search space, and then apply a memory-efficient ZO optimizer.

Table 2. Comparison of mainstream BP-free algorithms. **Parad.** (Paradigm): Gradient-Based (GB) and Gradient-Free (GF). **Signal:** T = Task (PG=Proxy Grad, FG=Forward Grad, ZO=Zeroth-Order); S = Synthetic (AS=Act. Stats, TP=Target Prop, SG=Synthetic Grad); H-S = Hebbian; E-S = Evolutionary. **Unlock:** FU = Forward Unlocking; BU = Backward Unlocking. **Bio.** / **Mem.**: Biological Plausibility and Memory Efficiency (● = High, ◐ = Medium, ○ = Low). **Perf. vs BP:** Accuracy difference relative to standard BP. **Code: Yes** = Official source; **Yes*** = Recommended unofficial/community implementation; No = Not available. Clicking on Yes or Yes* will open the corresponding external website.

| Method | Parad. | Signal | Unlock | Bio. | Mem. | Perf. vs BP | Task | Limitations | Code |
|---|---|---|---|---|---|---|---|---|---|
| AM [145] | GB | T-PG | - | ◐ | ◐ | -18.0 ~ -2.44 | Image Classification | Computationally intensive local optimization | **Yes** |
| CHL [146] | GF | H-S | - | ● | ◐ | +1.5 | Image Recon. & Class. | Original formulation requires symmetric weights | No |
| rCHL [146] | GF | H-S | - | ● | ◐ | +1.3 | Image Recon. & Class. | Relies on alignment to a fixed random path | No |
| DNI [57] | GB | S-SG | BU | ◐ | ◐ | -7.7 ~ +1.0 | Image Class. & Seq. Modeling | Adds a trainable synthetic gradient model | Yes* |
| cDNI [57] | GB | S-SG | BU | ◐ | ◐ | -5.0 ~ +0.2 | Image Class. & Seq. Modeling | Still adds parametric complexity | Yes* |
| DTP [107] | GB | S-TP | - | ◐ | ◐ | -1.59 ~ +0.31 | Image Classification | Inverse model training can be unstable | **Yes** |
| DFA [56] | GB | T-PG | BU | ◐ | ◐ | -20.27 ~ +0.09 | Image Classification | Needs global error broadcast; sensitive feedback | Yes* |
| IFA [56] | GB | T-PG | - | ◐ | ◐ | ~ -2.0 | Image Classification | Sequential error propagation; performance lags | No |
| DRTP [103] | GB | T-PG | BU | ◐ | ◐ | -14.46 ~ -1.54 | Image Classification | Performance typically lags behind BP | **Yes** |
| PEPITA [92] | GB | T-PG | BU | ● | ◐ | -8.66 ~ -0.57 | Image Classification | Standard version's memory usage similar to BP | **Yes** |
| FA [55] | GB | T-PG | No | ◐ | ◐ | -16.08 ~ +0.22 | Image Classification | Performance degrades in very deep networks | No |
| FG [31] | GB | T-FG | - | ○ | ◐ | 0.0 | Image Classification | High variance in high-dimensional spaces | Yes* |
| DODGE [28] | GB | T-FG | BU | ◐ | ● | -6.0 ~ -1.0 | RL Control & Image Class. | High variance in high-dimensional spaces | No |
| FoMoH [33] | GB | T-FG | BU | ◐ | ● | -2.31 ~ -1.86 | Regression & Image Class. | Hyper-dual overhead; inversion cost is substantial | No |
| FF [82] | GB | S-AS | BU | ● | ● | -1.34 ~ +0.19 | Image Classification | Needs negative data & two forward passes | **Yes** |
| PFF [144] | GB | S-AS | FU&BU | ● | ● | -0.06 ~ +0.32 | Image Gen. & Classification | Complex dual-circuit architecture | **Yes** |
| PredSim [61] | GB | T-PG | FU&BU | ◐ | ● | -0.28 ~ +0.97 | Image Classification | Needs local labels/projection; auxiliary nets | **Yes** |
| SID [11] | GB | T-PG | BU | ◐ | ● | -0.10 ~ +6.86 | Image Classification | Module-wise architecture constraints | **Yes** |
| ReduNet [84] | GB | T-FG | BU | ◐ | ◐ | -21.55 ~ +2.94 | White-box Image Class. | Eextreme localization & specialized architecture | **Yes** |
| HSIC Bneck [147] | GB | T-PG | BU | ◐ | ○ | -5.0 ~ +2.9 | Image Classification | Costly kernel matrix computation | **Yes** |
| KP [60] | GB | T-PG | - | ◐ | ◐ | -16.18 ~ -0.04 | Image Classification | Requires specific reciprocal network structures | **Yes** |
| KP-WM [60] | GB | T-PG | - | ◐ | ◐ | -1.0 ~ +0.9 | Image Classification | Requires specific reciprocal network structures | **Yes** |
| Mono-Fwd [85] | GB | T-PG | BU | ● | ● | 0.0 ~ +12.67 | Image Classification | Greedy layer-wise approach may be suboptimal | No |
| NoProp [58] | GB | T-PG | FU&BU | ○ | ◐ | -5.61 ~ +0.96 | Image Classification | Slow, sequential inference process | **Yes** |
| PFA [148] | GB | T-PG | - | ◐ | ◐ | -1.23 ~ +0.1 | Image Classification | Increases model size with intermediate layer | No |
| PC [16] | GB | S-TP | FU&BU | ● | ● | +0.59 | Image Recon. & Synthesis | Inference requires a slow settling process | Yes* |
| $\mu$PC [65] | GB | S-TP | FU&BU | ◐ | ● | -1.3 ~ 0.0 | Image Classification | Inference ill-conditioning grows with depth | No |
| Z-IL [64] | GB | S-TP | BU | ◐ | ● | 0.0 | Image Classification | Exact BP equivalent; requires strict synchronization | No |
| rec-LRA [73] | GB | S-TP | BU | ◐ | ◐ | -11.11 ~ +1.11 | Image Classification | Error-path topology can be complex to design | No |
| SS [149] | GB | T-PG | - | ◐ | ◐ | -5.06 ~ +3.12 | Image Classification | Robustness via gradient obfuscation circumvents | **Yes** |
| SoftHebb [105] | GF | H-S | BU | ● | ● | -11.8 ~ +1.72 | Unsup. Feature Learning | Sensitive to temperature parameter | **Yes** |
| Kickback [102] | GB | T-PG | - | ● | ◐ | 0.0 | Regression & Classification | Requires network "coherence" for convergence | No |
| NGC [150] | GF | H-S | FU&BU | ● | ● | -7.36~ +2.02 | Image Gen. & Recon. | Inference requires settling to equilibrium | No |
| RGE [43] | GB | T-ZO | BU | ○ | ● | / | Black-box Optimization | Complexity scales with dimension; high variance | No |
| MeZO [45] | GB | T-ZO | BU | ○ | ● | -6.1~ -0.6 | Language Class. & Gen. | High gradient variance leads to instability | **Yes** |
| ZO-AdaMU [53] | GB | T-ZO | BU | ○ | ● | -9.4~ +10.0 | Language Class. & Gen. | More complex than MeZO; more hyperparameters | **Yes** |
| DeepZero [47] | GB | T-ZO | BU | ○ | ● | -5.41 ~ -2.4 | Image Classification | High query complexity; relies on sparsity | **Yes** |
| ASAP [151] | GB | T-PG | - | ◐ | ◐ | -25.62 ~ -1.21 | Image Classification | "Activation sharing" limits performance | **Yes** |
| DGL [74] | GB | T-PG | BU | ◐ | ● | -0.03 ~ +1.2 | Image Classification | Performance relies on local classifier quality | **Yes** |
| WM [60] | GF | H-S | - | ◐ | ◐ | -0.5~ -0.1 | Image Classification | Requires separate learning phase | **Yes** |

The demonstrated success of FwdLLM [153], which integrates these components into a systematic framework, further validates this approach as the current state-of-the-art for on-device and black-box LLM adaptation.

**Building Scalable, Parallel, and Asynchronous Training Systems.** In scenarios where the goal is to maximize training throughput on large-scale distributed systems or to design hardware-friendly parallel architectures, algorithms based on local greedy learning offer a compelling alternative to the synchronous lock-step nature of BP. Methods like DGL [74] and DNI [57] break the "update locking" problem by decomposing the network into modules that can be trained independently and asynchronously. This is particularly relevant for model parallelism, where different parts of a large model are placed on different accelerators. Instead of complex pipeline parallelism required by BP, these methods allow each accelerator to update its local module without waiting for a full forward-backward pass through the entire model, potentially leading to significant speedups. While these methods introduce the challenge of aligning local and global objectives, their proven performance on benchmarks like ImageNet suggests they are a viable and powerful strategy. For systems architects, exploring hardware and software co-designs that can fully exploit the asynchrony and locality of these algorithms, such as specialized interconnects or memory architectures, is a highly recommended direction. The NoProp [58] framework, with its extreme parallelism of training each module as

Table 3. Overview of domain-specific BP-free applications. **Field:** The primary domain of application. **Task:** The specific problem addressed within the field.

| Method | Parad. | Signal | Unlock | Bio. | Mem. | Field | Task | Limitations |
|---|---|---|---|---|---|---|---|---|
| SubZero [127] | GB | T-ZO | - | ○ | ● | LLMs | Fine-Tuning | heavy computation overhead; slow convergence |
| BBT [121] | GF | E-S | FU&BU | ○ | ● | LLMs | Black-Box Prompting | Subspace optimality relies on random projection |
| BBTv2 [122] | GF | E-S | FU&BU | ○ | ● | LLMs | Black-Box Prompting | Sequential layer-wise optimization can be slow |
| HyperTuning [152] | GB | S-TP | FU&BU | ○ | ● | LLMs | Adaptation / Few-Shot | Limited context; requires training hypermodel |
| Clip-Tuning [123] | GF | E-S | FU&BU | ○ | ● | LLMs | Black-Box Prompting | Requires API support for dropout masks |
| FwdLLM [153] | GB | T-ZO | - | ○ | ● | LLMs | Federated Fine-Tuning | High system complexity |
| GRIPS [154] | GF | E-S | FU&BU | ○ | ● | LLMs | Discrete Prompting | Local search can get stuck in local optima |
| Promptboosting [155] | GF | E-S | FU&BU | ○ | ● | LLMs | Prompting | Optimizes verbalizer, not prompt |
| BDPL [156] | GF | E-S | FU&BU | ○ | ● | LLMs | Black-Box Prompting | Policy gradient has high variance |
| FedFwd [119] | GB | S-AS | BU | ◐ | ● | FL | On-Device Training | Dependence on negative data in local FF training |
| BAFFLE [104] | GB | T-ZO | - | ○ | ● | FL | On-Device Training | Communication cost scales with perturbations |
| LAFD [120] | GB | T-PG | - | ◐ | ● | FL | On-Device Training | Complex framework combining multiple methods |
| TIFeD [118] | GB | T-PG | - | ◐ | ● | FL | On-Device Training | Single-layer mode creates coordination challenges |
| MEMPEPITA [128] | GB | T-PG | - | ● | ● | FL | On-Device Training | Increases computational cost (3 forward passes) |
| Burstprop [93] | GB | T-PG | - | ● | ◐ | Neuromorphic | Temporal Learning | Relies on specific neuronal burst dynamics |
| e-prop [17] | GB | T-PG | BU | ● | ◐ | Neuromorphic | Temporal Learning | Approximate BPTT; poor long-sequence performance |
| Spiking FF [117] | GB | S-AS | BU | ◐ | ● | Neuromorphic | Supervised SNN | Dependence on negative data in local FF training |
| WSG-RFG-G [34] | GB | T-FG | BU | ◐ | ● | Neuromorphic | SciML & SNNs | High variance; large performance gap from BP |
| EqSpike [131] | GB | S-TP | FU&BU | ● | ◐ | Neuromorphic | Hardware Implementation | Requires specialized circuits for derivatives |
| SigProp [106] | GB | S-TP | BU | ● | ● | Neuromorphic | Hardware Implementation | Doubles channel width at each layer |
| DECOLLE [75] | GB | T-PG | BU | ● | ● | Neuromorphic | SNN Learning | Performance depends on random readouts |
| HEE [157] | GB | H-S | FU&BU | ● | ● | Neuromorphic | SNN Learning | Relies on complex dual-timescale neuron dynamics |
| MOHQA [113] | GF | H-S | - | ● | ◐ | RL | Reinforcement Learning | Hybrid architecture; not fully BP-free |
| RTRRL [112] | GB | T-PG | BU | ● | ◐ | RL | Real-Time Learning | Approximations miss very long dependencies |
| ANGC [158] | GB | S-TP | FU&BU | ● | ● | RL | Reinforcement Learning | Complex dual-circuit; iterative inference |
| SRC-EP [116] | GB | S-TP | FU&BU | ● | ◐ | Continual | Catastrophic Forgetting | Requires separate "sleep" phases |
| FoRo [130] | GF | E-S | BU | ○ | ● | Continual | On-Device Adaptation | Uses CMA-ES (expensive) and RLS (high memory) |
| DFA-Cont. [114] | GB | T-PG | - | ● | ◐ | Continual | Catastrophic Forgetting | Feedback effectiveness is scenario-dependent |
| HEB [142] | GB | S-TP | BU | ◐ | ● | Physical | Physical Learning | Requires time-reversal & dissipation control |
| ODFA [135] | GB | T-PG | - | ◐ | ◐ | Physical | Photonic Hardware | Requires specialized photonic hardware |
| PINN-Phot. [136] | GB | T-ZO | - | ○ | ● | Physical | Photonic Hardware | Relies on tensor compression, not optimal for all |
| QEP [143] | GB | S-TP | FU&BU | ○ | ◐ | Quantum | Quantum Hardware | Requires preparing quantum ground states |
| FORWARDGNN [86] | GB | T-PG | BU | ◐ | ● | Graph | Graph Learning | Dependence on virtual nodes for graph restructuring |

an independent denoising task, represents the theoretical limit of this decoupling and is particularly suited for future massively parallel or unconventional computing substrates.

**Achieving State-of-the-Art Performance While Maintaining Biological Plausibility.** For researchers aiming to bridge the gap between AI and neuroscience, or to build high-performance systems that adhere more closely to biological constraints, methods that learn the feedback pathway represent the most promising research frontier. While fixed-feedback methods like DFA [56] are conceptually simple, their performance limitations on complex tasks are significant. In contrast, algorithms like the improved kolen-pollack algorithm, weight mirror [60], ASAP [151], and PFA [148] have all demonstrated the ability to achieve performance nearly identical to BP on ImageNet. These methods work by establishing a dynamic process where the feedback weight matrix is continuously updated to align with the transpose of the feedforward weight matrix, effectively solving the weight transport problem without requiring a biologically implausible direct copy. Among these, the KP [60] algorithm and its variants are particularly elegant as they emerge from a simple mathematical principle: two variables will converge if their update rules are identical. While these methods introduce additional computational steps or phases for learning the feedback weights, their success proves that functional symmetry can be learned rather than hardwired. For researchers in this domain, a key next step is to investigate how these alignment mechanisms interact with other biological features, such as sparse activations and event-driven communication (spikes), to build even more realistic and efficient learning systems. EP [76] and its variants [76, 111, 143], which are mathematically guaranteed to compute exact gradients under certain conditions, also fall into this category and are highly recommended for applications requiring both high performance and a strong physical or theoretical grounding.

**Low-Power Event-Driven Neuromorphic Hardware.** When the target platform is neuromorphic hardware, which is inherently designed for sparse event-driven and local computation, algorithms based on local Hebbian plasticity and predictive coding are the natural and recommended choice. Standard BP is extremely ill-suited for such hardware. In contrast, algorithms like SoftHebb [105], with its purely local, unsupervised, and feedback-free learning rule, can be directly mapped to the physical constraints of neuromorphic chips. Similarly, predictive coding frameworks like NGC [150] and its RL extension ANGC [158], which operate via local prediction error minimization, align perfectly with the message-passing nature of neuromorphic architectures. These algorithms often leverage spiking dynamics, as seen in EqSpike [131] (a spiking version of EP), which not only makes them hardware-compatible but also allows them to naturally process sparse temporal data. The key advantage here is not necessarily outperforming BP on static image benchmarks, but enabling continuous online learning in real-world sensory-motor loops with ultra-low power consumption. Therefore, for the neuromorphic community, focusing on co-designing hardware primitives that efficiently implement these local error-driven Hebbian updates is the most promising path toward truly intelligent edge devices.

## 9 Future Directions

While considerable progress has been achieved, BP-free learning still faces fundamental challenges that limit its theoretical understanding, scalability, and practical applicability. In this context, several key open problems have emerged that define the current boundaries of research and highlight directions for future investigation.

**Theoretical Understanding.** While practical advances in BP-free training have been substantial, their theoretical foundations remain largely underexplored. For gradient-free approaches, the absence of explicit gradient computation makes it difficult to guarantee convergence, and early paradigms such as Hebbian learning [87] often suffer from unstable or divergent behaviors. For gradient-based BP-free methods, most works focus on reducing the discrepancy between the estimated and true gradients [11, 57], yet few provide formal convergence analysis—an aspect that is crucial for understanding optimization stability. Only a limited subset of BP-free algorithms, such as zeroth-order optimization, have established convergence results, but even these rarely address generalization behavior. From a learning-theoretic perspective, these limitations raise a fundamental question: *are current BP-free algorithms probably approximately correct (PAC)-learnable?* PAC learning [160] formalizes the notion that, given a finite dataset, a learning algorithm is expected to output a hypothesis that is approximately optimal with high probability. Without provable guarantees on convergence and generalization, BP-free training cannot yet be shown to satisfy this criterion. Therefore, a promising theoretical direction is to examine BP-free learning through the lens of PAC learnability, neural dynamical stability, and probabilistic convergence, which may together shed light on how such systems can achieve stable learning.

**Scalability to Large and Deep Models.** Most existing BP-free studies have been limited to relatively small-scale architectures such as AlexNet or shallow CNNs. Only a few optimization paradigms, such as zeroth-order optimization and evolutionary strategies, have been applied to large-scale models, yet their performance still lags far behind standard BP [45, 130]. This limitation largely stems from the poor scalability of existing BP-free mechanisms, whose computational cost often increases rapidly with model depth. For instance, in DNI [57], the number of proxy gradient estimators increases linearly with the number of layers, while NoProp [58] incurs a diffusion step whose cost also scales with depth. Similarly, in zeroth-order optimization, the variance of gradient estimates grows with model dimensionality, making it increasingly difficult to obtain stable updates as models become larger. Such growth in auxiliary components and estimation variance severely limits the applicability of BP-free methods to deep or large models. Addressing this limitation calls for scalable BP-free algorithms that explicitly account for the coupling between algorithmic efficiency

and model size. Promising directions may involve hierarchical gradient synthesis, parameter-sharing estimators, or hybrid designs that combine BP-free modules with partial BP to achieve both efficiency and scalability.

**Balancing Biological Plausibility, Computational Performance, and Memory Efficiency.** Given the exceptional performance of BP, approaches aiming for high accuracy frequently compromise biological plausibility or memory efficiency—and vice versa [161]. Achieving a balanced trade-off among these three aspects remains an open challenge in BP-free learning. A key difficulty lies in jointly quantifying these objectives in a unified optimization framework, as they involve fundamentally different metrics—biological plausibility is qualitative and mechanism-oriented, whereas performance and memory efficiency are quantitative and system-dependent. To address this challenge, one promising direction is to establish a compatible multi-objective trade-off surface that enables these objectives to be jointly represented and compared. Such a surface would provide a principled basis for guiding architecture search, where an intriguing question is *how a neural architecture could be searched or adapted to occupy arbitrary points on this trade-off surface?* One possible solution is to leverage reinforcement learning with human feedback (RLHF) [162] or other preference-based optimization schemes to dynamically explore the multi-objective trade-off space, enabling the automated discovery of biologically grounded yet computationally efficient learning architectures.

**Incompatibility of Biologically Inspired Learning with ANN Architectures.** Current BP-free algorithms are mostly implemented and evaluated on standard artificial neural networks (ANNs). However, these architectures were primarily developed to support BP-based training, rather than to reflect biological learning mechanisms. As a result, BP-free algorithms are often constrained by the standard network architectures commonly used in practice, which can limit their performance and prevent them from fully realizing biologically plausible learning. From a neuro-inspired perspective, the incompatibility between BP-free learning rules and popular ANN architectures raises a deeper question: *Can biologically plausible learning truly emerge on networks that are not biologically structured?* Real neural systems leverage diverse signaling dynamics—spikes, local dendritic processing, and neuromodulatory feedback—that are absent in conventional ANNs. Without such mechanisms, BP-free algorithms often end up being "approximations of BP" rather than fundamentally new learning paradigms. Therefore, advancing BP-free learning may require rethinking the computational substrate itself, moving from static ANN architectures toward neuromorphic or dynamically structured networks that better capture biological communication and local learning. Future research could utilize abstract representations of the brain's hierarchical cortical layers to guide the modeling of brain-like neural network architectures [163].

**Hardware Support for Neuromorphic Computing.** Currently, most BP-free algorithms are widely known to underperform compared to traditional BP. However, this does not imply that BP-free algorithms are inherently inferior; rather, it may reflect the lack of adequate hardware support. Spiking neural networks (SNNs), characterized by neuron-like event-driven dynamics, have already shown promising potential, achieving up to 87× lower energy consumption compared to conventional BP-trained ANNs [164]. Although neuromorphic hardware such as SpiNNaker [165] and IBM TrueNorth [22] has provided partial support for SNNs, more powerful and better-matched hardware is still lacking. As noted by Kudithipudi et al. [163], neuromorphic computing is currently awaiting an "AlexNet-like moment"—a breakthrough small-scale system that could catalyze large-scale neuromorphic computing development. Therefore, future research should focus on two complementary directions: (i) developing BP-free algorithms and neural network architectures that effectively utilize current neuromorphic hardware, and (ii) designing and implementing hardware that efficiently supports neuromorphic computation, allowing SNNs and other BP-free algorithms to fully exploit their energy-efficient potential and scale to more complex applications.

Building on these insights, BP-free learning requires integrating theoretical understanding, scalable algorithmic designs, biologically grounded principles, and supportive neuromorphic hardware into a coherent research agenda. By highlighting current limitations and potential directions, we aim to guide future efforts toward more robust, efficient, and biologically plausible learning paradigms.

## References


[1] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[2] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[3] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.

[5] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

[6] Siddique Latif, Moazzam Shoukat, Fahad Shamshad, Muhammad Usama, Yi Ren, Heriberto Cuayáhuitl, Wenwu Wang, Xulong Zhang, Roberto Togneri, Erik Cambria, et al. Sparks of large audio models: A survey and outlook. *arXiv preprint arXiv:2308.12792*, 2023.

[7] Sergey Bartunov, Adam Santoro, Blake Richards, Luke Marris, Geoffrey E Hinton, and Timothy Lillicrap. Assessing the scalability of biologically-motivated deep learning algorithms and architectures. In *Proceedings of Advances in Neural Information Processing Systems*, volume 31, 2018.

[8] Stephen Grossberg. Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1):23–63, 1987.

[9] Nasir Ahmad, Marcel A van Gerven, and Luca Ambrogioni. GAIT-prop: A biologically plausible learning rule derived from backpropagation of error. In *Proceedings of Advances in Neural Information Processing Systems*, volume 33, pages 10913–10923, 2020.

[10] Zhouyuan Huo, Bin Gu, Heng Huang, et al. Decoupled parallel backpropagation with convergence guarantee. In *Proceedings of International Conference on Machine Learning*, pages 2098–2106. PMLR, 2018.

[11] Chenhao Ye and Ming Tang. Learning without global backpropagation via synergistic information distillation. *arXiv preprint arXiv:2510.03273*, 2025.

[12] Pedro O Domingos, Fernando M Silva, and Horácio C Neto. An efficient and scalable architecture for neural networks with backpropagation learning. In *Proceedings of International Conference on Field Programmable Logic and Applications*, pages 89–94. IEEE, 2005.

[13] Sijia Liu, Pin-Yu Chen, Bhavya Kailkhura, Gaoyuan Zhang, Alfred O Hero III, and Pramod K Varshney. A primer on zeroth-order optimization in signal processing and machine learning: Principals, recent advances, and applications. *IEEE Signal Processing Magazine*, 37(5):43–54, 2020.

[14] Shiyu Duan and Jose C Principe. Training deep architectures without end-to-end backpropagation: A survey on the provably optimal methods. *IEEE Computational Intelligence Magazine*, 17(4):39–51, 2022.

[15] Wulfram Gerstner. Hebbian learning and plasticity. *From Neuron to Cognition via Computational Neuroscience*, pages 0–25, 2011.

[16] Rajesh PN Rao and Dana H Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.

[17] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11(1):3625, 2020.

[18] Anzhe Cheng, Heng Ping, Zhenkun Wang, Xiongye Xiao, Chenzhong Yin, Shahin Nazarian, Mingxi Cheng, and Paul Bogdan. Unlocking deep learning: A BP-free approach for parallel block-wise training of neural networks. In *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 4235–4239. IEEE, 2024.

[19] Anzhe Cheng, Chenzhong Yin, Mingxi Cheng, Shukai Duan, Shahin Nazarian, and Paul Bogdan. Eigen neural network: Unlocking generalizable vision with eigenbasis. *arXiv preprint arXiv:2508.01219*, 2025.

[20] Yihua Zhang, Pingzhi Li, Junyuan Hong, Jiaxiang Li, Yimeng Zhang, Wenqing Zheng, Pin-Yu Chen, Jason D Lee, Wotao Yin, Mingyi Hong, et al. Revisiting zeroth-order optimization for memory-efficient LLM fine-tuning: A benchmark. In *Proceedings of International Conference on Machine Learning*, pages 59173–59190. PMLR, 2024.

[21] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.

[22] Germain Haessig, Andrew Cassidy, Rodrigo Alvarez, Ryad Benosman, and Garrick Orchard. Spiking optical flow for event-based sensors using ibm's truenorth neurosynaptic system. *IEEE Transactions on Biomedical Circuits and Systems*, 12(4):860–870, 2018.

[23] Gabriele Lagani, Fabrizio Falchi, Claudio Gennaro, and Giuseppe Amato. Spiking neural networks and bio-inspired supervised deep learning: A survey, 2023.

[24] Hanzi Mei, Dongqi Cai, Yaozong Wu, Shangguang Wang, and Mengwei Xu. A survey of backpropagation-free training for llms. *Authorea Preprints*, 2024.

[25] Björn van Zwol, Ro Jefferson, and Egon L Broek. Predictive coding networks and inference learning: Tutorial and survey. *arXiv preprint arXiv:2407.04117*, 2024.

[26] Tommaso Salvatori, Ankur Mali, Christopher L Buckley, Thomas Lukasiewicz, Rajesh PN Rao, Karl Friston, and Alexander Ororbia. A survey on neuro-mimetic deep learning via predictive coding. *Neural Networks*, page 108161, 2025.

[27] Michiel Hermans, Michaël Burm, Thomas Van Vaerenbergh, Joni Dambre, and Peter Bienstman. Trainable hardware for dynamical computing using error backpropagation through physical media. *Nature Communications*, 6(1):6729, 2015.

[28] David Silver, Anirudh Goyal, Ivo Danihelka, Matteo Hessel, and Hado van Hasselt. Learning by directional gradient descent. In *Proceedings of International Conference on Learning Representations*, 2021.

[29] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.

[30] Robert Edwin Wengert. A simple automatic derivative evaluation program. *Communications of the ACM*, 7(8):463–464, 1964.

[31] Atılım Güneş Baydin, Barak A Pearlmutter, Don Syme, Frank Wood, and Philip Torr. Gradients without backpropagation. *arXiv preprint arXiv:2202.08587*, 2022.

[32] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, et al. Jax: Composable transformations of python+ numpy programs. 2018.

[33] Adam D Cobb, Atilim Gunes Baydin, Barak A Pearlmutter, and Susmit Jha. Second-order forward-mode automatic differentiation for optimization. In *Proceedings of Optimization for Machine Learning*, 2024.

[34] Ruyin Wan, Qian Zhang, and George Em Karniadakis. Randomized forward mode gradient for spiking neural networks in scientific machine learning. *arXiv preprint arXiv:2411.07057*, 2024.

[35] Katharina Flügel, Daniel Coquelin, Marie Weiel, Achim Streit, and Markus Götz. Beyond backpropagation: Optimization with multi-tangent forward gradients. *arXiv preprint arXiv:2410.17764*, 2024.

[36] Ila R Fiete and H Sebastian Seung. Gradient learning in spiking neural networks by dynamic perturbation of conductances. *Physical Review Letters*, 97(4):048104, 2006.

[37] Mengye Ren, Simon Kornblith, Renjie Liao, and Geoffrey Hinton. Scaling forward gradient with local losses. In *Proceedings of International Conference on Learning Representations*, 2023.

[38] Eugene Belilovsky, Louis Leconte, Lucas Caccia, Michael Eickenberg, and Edouard Oyallon. Decoupled greedy learning of CNNs for synchronous and asynchronous distributed learning. *arXiv preprint arXiv:2106.06401*, 2021.

[39] Louis Fournier, Stéphane Rivaud, Eugene Belilovsky, Michael Eickenberg, and Edouard Oyallon. Can forward gradient match backpropagation? In *Proceedings of International Conference on Machine Learning*, pages 10249–10264. PMLR, 2023.

[40] Daniel Wang, Evan Markou, and Dylan Campbell. Towards scalable backpropagation-free gradient estimation. *arXiv preprint arXiv:2511.03110*, 2025.

[41] Abraham D Flaxman, Adam Tauman Kalai, and H Brendan McMahan. Online convex optimization in the bandit setting: Gradient descent without a gradient. In *Proceedings of Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 385–394, 2005.

[42] Yurii Nesterov and Vladimir Spokoiny. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017.

[43] Saeed Ghadimi and Guanghui Lan. Stochastic first-and zeroth-order methods for nonconvex stochastic programming. *SIAM journal on optimization*, 23(4):2341–2368, 2013.

[44] John C Duchi, Michael I Jordan, Martin J Wainwright, and Andre Wibisono. Optimal rates for zero-order convex optimization: The power of two function evaluations. *IEEE Transactions on Information Theory*, 61(5):2788–2806, 2015.

[45] Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D Lee, Danqi Chen, and Sanjeev Arora. Fine-tuning language models with just forward passes. In *Proceedings of Advances in Neural Information Processing Systems*, volume 36, pages 53038–53075, 2023.

[46] Sijia Liu, Bhavya Kailkhura, Pin-Yu Chen, Paishun Ting, Shiyu Chang, and Lisa Amini. Zeroth-order stochastic variance reduction for nonconvex optimization. In *Proceedings of Advances in Neural Information Processing Systems*, volume 31, 2018.

[47] Aochuan Chen, Yimeng Zhang, Jinghan Jia, James Diffenderfer, Konstantinos Parasyris, Jiancheng Liu, Yihua Zhang, Zheng Zhang, Bhavya Kailkhura, and Sijia Liu. Deepzero: Scaling up zeroth-order optimization for deep model training. In *Proceedings of International Conference on Learning Representations*, 2024.

[48] Shuyu Cheng, Guoqiang Wu, and Jun Zhu. On the convergence of prior-guided zeroth-order optimization algorithms. In *Proceedings of Advances in Neural Information Processing Systems*, volume 34, pages 14620–14631, 2021.

[49] Florian Meier, Asier Mujika, Marcelo Matheus Gauy, and Angelika Steger. Improving gradient estimation in evolutionary strategies with past descent directions. *arXiv preprint arXiv:1910.05268*, 2019.

[50] Yining Wang, Simon Du, Sivaraman Balakrishnan, and Aarti Singh. Stochastic zeroth-order optimization in high dimensions. In *Proceedings of International Conference on Artificial Intelligence and Statistics*, pages 1356–1365. PMLR, 2018.

[51] HanQin Cai, Daniel McKenzie, Wotao Yin, and Zhenliang Zhang. Zeroth-order regularized optimization (ZORO): Approximately sparse gradients and adaptive sampling. *SIAM Journal on Optimization*, 32(2):687–714, 2022.

[52] Bumsu Kim, Daniel McKenzie, Hanqin Cai, and Wotao Yin. Curvature-aware derivative-free optimization. *Journal of Scientific Computing*, 103(2):43, 2025.

[53] Shuoran Jiang, Qingcai Chen, Youcheng Pan, Yang Xiang, Yukang Lin, Xiangping Wu, Chuanyi Liu, and Xiaobao Song. Zo-Adamu optimizer: Adapting perturbation by the momentum and uncertainty in zeroth-order optimization. In *Proceedings of the AAAI Conference on Artificial*

*Intelligence*, volume 38, pages 18363–18371, 2024.

[54] Zhen Qin, Daoyuan Chen, Bingchen Qian, Bolin Ding, Yaliang Li, and Shuiguang Deng. Federated full-parameter tuning of billion-sized language models with communication cost under 18 kilobytes. In *Proceedings of International Conference on Machine Learning*, pages 41473–41497. PMLR, 2024.

[55] Timothy P Lillicrap, Daniel Cownden, Douglas B Tweed, and Colin J Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7(1):13276, 2016.

[56] Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Proceedings of Advances in Neural Information Processing Systems*, volume 29, 2016.

[57] Max Jaderberg, Wojciech Marian Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Decoupled neural interfaces using synthetic gradients. In *Proceedings of International Conference on Machine Learning*, pages 1627–1635. PMLR, 2017.

[58] Qinyu Li, Yee Whye Teh, and Razvan Pascanu. Noprop: Training neural networks without back-propagation or forward-propagation. *arXiv preprint arXiv:2503.24322*, 2025.

[59] James M Murray. Local online learning in recurrent networks with random feedback. *Elife*, 8:e43299, 2019.

[60] Mohamed Akrout, Collin Wilson, Peter Humphreys, Timothy Lillicrap, and Douglas B Tweed. Deep learning without weight transport. *Proceedings of Advances in Neural Information Processing Systems*, 32, 2019.

[61] Arild Nøkland and Lars Hiller Eidnes. Training neural networks with local error signals. In *Proceedings of International Conference on Machine Learning*, pages 4839–4850. PMLR, 2019.

[62] James CR Whittington and Rafal Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local hebbian synaptic plasticity. *Neural Computation*, 29(5):1229–1262, 2017.

[63] Beren Millidge, Alexander Tschantz, and Christopher L Buckley. Predictive coding approximates backprop along arbitrary computation graphs. *Neural Computation*, 34(6):1329–1368, 2022.

[64] Tommaso Salvatori, Yuhang Song, Zhenghua Xu, Thomas Lukasiewicz, and Rafal Bogacz. Reverse differentiation via predictive coding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8150–8158, 2022.

[65] Francesco Innocenti, El Mehdi Achour, and Christopher L Buckley. $\mu$PC: Scaling predictive coding to 100+ layer networks. *arXiv preprint arXiv:2505.13124*, 2025.

[66] Blake Bordelon, Lorenzo Noci, Mufan Bill Li, Boris Hanin, and Cengiz Pehlevan. Depthwise hyperparameter transfer in residual networks: Dynamics and scaling limit. In *Proceedings of International Conference on Learning Representations*, 2024.

[67] Chang Qi, Matteo Forasassi, Thomas Lukasiewicz, and Tommaso Salvatori. Towards the training of deeper predictive coding neural networks. *arXiv preprint arXiv:2506.23800*, 2025.

[68] Chang Qi, Thomas Lukasiewicz, and Tommaso Salvatori. Training deep predictive coding networks. In *Proceedings of New Frontiers in Associative Memories*, 2025.

[69] Luca Pinchetti, Tommaso Salvatori, Yordan Yordanov, Beren Millidge, Yuhang Song, and Thomas Lukasiewicz. Predictive coding beyond gaussian distributions. In *Proceedings of Advances in Neural Information Processing Systems*, pages 1280–1293, 2022.

[70] Eli Sennesh, Hao Wu, and Tommaso Salvatori. Divide-and-conquer predictive coding: a structured bayesian inference algorithm. In *Proceedings of Advances in Neural Information Processing Systems*, volume 37, pages 48599–48627, 2024.

[71] Yoshua Bengio. How auto-encoders could provide credit assignment in deep networks via target propagation. *arXiv preprint arXiv:1407.7906*, 2014.

[72] Dong-Hyun Lee, Saizheng Zhang, Asja Fischer, and Yoshua Bengio. Difference target propagation. In *Proceedings of Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 498–515. Springer, 2015.

[73] Alexander G Ororbia, Ankur Mali, Daniel Kifer, and C Lee Giles. Backpropagation-free deep learning with recursive local representation alignment. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 9327–9335, 2023.

[74] Eugene Belilovsky, Michael Eickenberg, and Edouard Oyallon. Decoupled greedy learning of CNNs. In *Proceedings of International Conference on Machine Learning*, pages 736–745. PMLR, 2020.

[75] Jacques Kaiser, Hesham Mostafa, and Emre Neftci. Synaptic plasticity dynamics for deep continuous local learning (DECOLLE). *Frontiers in Neuroscience*, 14:424, 2020.

[76] Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017.

[77] Javier R Movellan. Contrastive hebbian learning in the continuous hopfield model. In *Connectionist Models*, pages 10–17. Elsevier, 1991.

[78] Nicolas Zucchet and Joao Sacramento. Beyond backpropagation: Bilevel optimization through implicit differentiation and equilibrium propagation. *Neural Computation*, 34(12):2309–2346, 2022.

[79] Jack Kendall, Ross Pantone, Kalpana Manickavasagam, Yoshua Bengio, and Benjamin Scellier. Training end-to-end analog neural networks with equilibrium propagation. *arXiv preprint arXiv:2006.01981*, 2020.

[80] Benjamin Scellier. Quantum equilibrium propagation: Gradient-descent training of quantum systems. In *Proceedings of NeurIPS Workshop Machine Learning with New Compute Paradigms*, 2024.

[81] Timothy Nest and Maxence Ernoult. Towards training digitally-tied analog blocks via hybrid gradient computation. In *Proceedings of Advances in Neural Information Processing Systems*, volume 37, pages 83877–83914, 2024.

[82] Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2(3):5, 2022.

[83] Kwan Ho Ryan Chan, Yaodong Yu, Chong You, Haozhi Qi, John Wright, and Yi Ma. ReduNet: A white-box deep network from the principle of maximizing rate reduction. *Journal of Machine Learning Research*, 23(114):1–103, 2022.

[84] Kwan Ho Ryan Chan, Yaodong Yu, Chong You, Haozhi Qi, John Wright, and Yi Ma. Deep networks from the principle of rate reduction. *arXiv preprint arXiv:2010.14765*, 2020.

[85] James Gong, Bruce Li, and Waleed Abdulla. Mono-forward: Backpropagation-free algorithm for efficient neural network training harnessing local errors. *arXiv preprint arXiv:2501.09238*, 2025.

[86] Namyong Park, Xing Wang, Antoine Simoulin, Shuai Yang, Grey Yang, Ryan Rossi, Puja Trivedi, and Nesreen Ahmed. Forward learning of graph neural networks. *arXiv preprint arXiv:2403.11004*, 2024.

[87] Richard Kempter, Wulfram Gerstner, and J Leo Van Hemmen. Hebbian learning and spiking neurons. *Physical Review E*, 59(4):4498, 1999.

[88] Natalia Caporale and Yang Dan. Spike timing–dependent plasticity: a hebbian learning rule. *Annual Review of Neuroscience*, 31(1):25–46, 2008.

[89] Nicolas Brunel. Hebbian learning of context in recurrent neural networks. *Neural Computation*, 8(8):1677–1710, 1996.

[90] Xiaohui Xie and H Sebastian Seung. Equivalence of backpropagation and contrastive hebbian learning in a layered network. *Neural Computation*, 15(2):441–454, 2003.

[91] Timoleon Moraitis, Dmitry Toichkin, Adrien Journé, Yansong Chua, and Qinghai Guo. Softhebb: Bayesian inference in unsupervised hebbian soft winner-take-all networks. *Neuromorphic Computing and Engineering*, 2(4):044017, 2022.

[92] Giorgia Dellaferrera and Gabriel Kreiman. Error-driven input modulation: Solving the credit assignment problem without a backward pass. In *Proceedings of International Conference on Machine Learning*, pages 4937–4955. PMLR, 2022.

[93] Alexandre Payeur, Jordan Guerguiev, Friedemann Zenke, Blake A Richards, and Richard Naud. Burst-dependent synaptic plasticity can coordinate learning in hierarchical circuits. *Nature Neuroscience*, 24(7):1010–1019, 2021.

[94] Xin Yao. A review of evolutionary artificial neural networks. *International Journal of Intelligent Systems*, 8(4):539–567, 1993.

[95] Martin Mandischer. A comparison of evolution strategies and backpropagation for neural network training. *Neurocomputing*, 42(1-4):87–117, 2002.

[96] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

[97] Xin Qiu, Yulu Gan, Conor F Hayes, Qiyao Liang, Elliot Meyerson, Babak Hodjat, and Risto Miikkulainen. Evolution strategies at scale: LLM fine-tuning beyond reinforcement learning. *arXiv preprint arXiv:2509.24372*, 2025.

[98] Caio Davi and Ulisses Braga-Neto. PSP-PINN: Physics-informed neural networks trained with particle swarm optimization. *arXiv preprint arXiv:2202.01943*, 2022.

[99] Ercan Engin Kuruoglu, Chun Lin Kuo, and Wai Kin Victor Chan. Sparse neural network optimization by simulated annealing. *Franklin Open*, 4:100037, 2023.

[100] Manolis Papadrakakis, Nikos D Lagaros, and Yiannis Tsompanakis. Structural optimization using evolution strategies and neural networks. *Computer Methods in Applied Mechanics and Engineering*, 156(1-4):309–333, 1998.

[101] Sreenivas Sremath Tirumala. Evolving deep neural networks using coevolutionary algorithms with multi-population strategy. *Neural Computing and Applications*, 32(16):13051–13064, 2020.

[102] David Balduzzi, Hastagiri Vanchinathan, and Joachim Buhmann. Kickback cuts backprop's red-tape: Biologically plausible credit assignment in neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 29, 2015.

[103] Charlotte Frenkel, Martin Lefebvre, and David Bol. Learning without feedback: Fixed random learning signals allow for feedforward training of deep neural networks. *Frontiers in Neuroscience*, 15:629892, 2021.

[104] Haozhe Feng, Tianyu Pang, Chao Du, Wei Chen, Shuicheng Yan, and Min Lin. Baffle: A baseline of backpropagation-free federated learning. In *Proceedings of European Conference on Computer Vision*, pages 89–109. Springer, 2024.

[105] Adrien Journé, Hector Garcia Rodriguez, Qinghai Guo, and Timoleon Moraitis. Hebbian deep learning without feedback. In *Proceedings of International Conference on Learning Representations*, 2023.

[106] Adam Kohan, Edward A Rietman, and Hava T Siegelmann. Signal propagation: The framework for learning and inference in a forward pass. *IEEE Transactions on Neural Networks and Learning Systems*, 35(6):8585–8596, 2023.

[107] Maxence M Ernoult, Fabrice Normandin, Abhinav Moudgil, Sean Spinney, Eugene Belilovsky, Irina Rish, Blake Richards, and Yoshua Bengio. Towards scaling difference target propagation by learning backprop targets. In *Proceedings of International Conference on Machine Learning*, pages 5968–5987. PMLR, 2022.

[108] Gavin Taylor, Ryan Burmeister, Zheng Xu, Bharat Singh, Ankit Patel, and Tom Goldstein. Training neural networks without gradients: A scalable admm approach. In *Proceedings of International Conference on Machine Learning*, pages 2722–2731. PMLR, 2016.

[109] Miguel Carreira-Perpinan and Weiran Wang. Distributed optimization of deeply nested systems. In *Proceedings of Artificial Intelligence and Statistics*, pages 10–19. PMLR, 2014.

[110] Yoshimasa Kubo, Eric Chalmers, and Artur Luczak. Combining backpropagation with equilibrium propagation to improve an actor-critic reinforcement learning framework. *Frontiers in Computational Neuroscience*, 16:980613, 2022.

[111] Maxence Ernoult, Julie Grollier, Damien Querlioz, Yoshua Bengio, and Benjamin Scellier. Equilibrium propagation with continual weight updates. *arXiv preprint arXiv:2005.04168*, 2020.

[112] Julian Lemmel and Radu Grosu. Real-time recurrent reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18189–18197, 2025.

[113] Pawel Ladosz, Eseoghene Ben-Iwhiwhu, Jeffery Dick, Nicholas Ketz, Soheil Kolouri, Jeffrey L Krichmar, Praveen K Pilly, and Andrea Soltoggio. Deep reinforcement learning with modulated hebbian plus Q-network architecture. *IEEE Transactions on Neural Networks and Learning Systems*, 33(5):2045–2056, 2021.

[114] Sara Folchini, Viplove Arora, and Sebastian Goldt. Exploring the potential of direct feedback alignment for continual learning. *Transactions on Machine Learning Research*, 2025.

[115] Timo Flesch, David G Nagy, Andrew Saxe, and Christopher Summerfield. Modelling continual learning in humans with hebbian context gating and exponentially decaying task signals. *PLoS Computational Biology*, 19(1):e1010808, 2023.

[116] Yoshimasa Kubo, Jean Erik Delanois, and Maxim Bazhenov. Toward lifelong learning in equilibrium propagation: Sleep-like and awake rehearsal for enhanced stability. *arXiv preprint arXiv:2508.14081*, 2025.

[117] Mohammadnavid Ghader, Saeed Reza Kheradpisheh, Bahar Farahani, and Mahmood Fazlali. Backpropagation-free spiking neural networks with the forward-forward algorithm, 2025.

[118] Luca Colombo, Alessandro Falcetta, and Manuel Roveri. TIFeD: a tiny integer-based federated learning algorithm with direct feedback alignment. In *Proceedings of International Conference on AI-ML Systems*, pages 1–8, 2023.

[119] Seonghwan Park, Dahun Shin, Jinseok Chung, and Namhoon Lee. Fedfwd: Federated learning without backpropagation. *arXiv preprint arXiv:2309.01150*, 2023.

[120] Kijung Jung, Incheol Baek, Soohyung Kim, and Yon Dohn Chung. LAFD: Local-differentially private and asynchronous federated learning with direct feedback alignment. *IEEE Access*, 11:86754–86769, 2023.

[121] Tianxiang Sun, Yunfan Shao, Hong Qian, Xuanjing Huang, and Xipeng Qiu. Black-box tuning for language-model-as-a-service. In *Proceedings of International Conference on Machine Learning*, pages 20841–20855. PMLR, 2022.

[122] Tianxiang Sun, Zhengfu He, Hong Qian, Yunhua Zhou, Xuanjing Huang, and Xipeng Qiu. BBTV2: Towards a gradient-free future with large language models. *arXiv preprint arXiv:2205.11200*, 2022.

[123] Yekun Chai, Shuohuan Wang, Yu Sun, Hao Tian, Hua Wu, and Haifeng Wang. Clip-tuning: Towards derivative-free prompt learning with a mixture of rewards. *arXiv preprint arXiv:2210.12050*, 2022.

[124] Qiushi Sun, Chengcheng Han, Nuo Chen, Renyu Zhu, Jingyang Gong, Xiang Li, and Ming Gao. Make prompt-based black-box tuning colorful: Boosting model generalization from three orthogonal perspectives. *arXiv preprint arXiv:2305.08088*, 2023.

[125] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. *arXiv preprint arXiv:2309.08532*, 2023.

[126] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *Proceedings of International Conference on Learning Representations*, 2023.

[127] Ziming Yu, Pan Zhou, Sike Wang, Jia Li, and Hua Huang. SubZero: Random subspace zeroth-order optimization for memory-efficient llm fine-tuning. *arXiv e-prints*, pages arXiv–2410, 2024.

[128] Danilo Pietro Pau and Fabrizio Maria Aymone. Suitability of forward-forward and pepita learning to mlcommons-tiny benchmarks. In *Proceedings of IEEE International Conference on Omni-layer Intelligent Systems*, pages 1–6, 2023.

[129] Danilo Pietro Pau and Fabrizio Maria Aymone. Forward learning of large language models by consumer devices. *Electronics*, 13(2):402, 2024.

[130] Jiao Chen, Jiayi He, Fangfang Chen, Zuohong Lv, and Jianhua Tang. Forward-only continual learning. In *Proceedings of ACM International Conference on Multimedia*, pages 4048–4057, 2025.

[131] Erwann Martin, Maxence Ernoult, Jérémie Laydevant, Shuai Li, Damien Querlioz, Teodora Petrisor, and Julie Grollier. Eqspike: Spike-driven equilibrium propagation for neuromorphic implementations. *Iscience*, 24(3), 2021.

[132] Peter O'Connor, Efstratios Gavves, and Max Welling. Training a spiking neural network with equilibrium propagation. In *Proceedings of International Conference on Artificial Intelligence and Statistics*, pages 1516–1523. PMLR, 2019.

[133] Brady Taylor, Nicky Ramos, Eric Yeats, and Hai Li. Cmos implementation of spiking equilibrium propagation for real-time learning. In *Proceedings of International Conference on Artificial Intelligence Circuits and Systems*, pages 283–286. IEEE, 2022.

[134] Julien Launay, Iacopo Poli, Kilian Müller, Gustave Pariente, Igor Carron, Laurent Daudet, Florent Krzakala, and Sylvain Gigan. Hardware beyond backpropagation: a photonic co-processor for direct feedback alignment. *arXiv preprint arXiv:2012.06373*, 2020.

[135] Ziao Wang, Kilian Müller, Matthew Filipovich, Julien Launay, Ruben Ohana, Gustave Pariente, Safa Mokaadi, Charles Brossollet, Fabien Moreau, Alessandro Cappelli, et al. Streamlined optical training of large-scale modern deep learning architectures with direct feedback alignment. *arXiv preprint arXiv:2409.12965*, 2024.

[136] Yequan Zhao, Xinling Yu, Xian Xiao, Zhixiong Chen, Ziyue Liu, Geza Kurczveil, Raymond G Beausoleil, Sijia Liu, and Zheng Zhang. Scalable back-propagation-free training of optical physics-informed neural networks. *arXiv preprint arXiv:2502.12384*, 2025.

[137] Saumil Bandyopadhyay, Alexander Sludds, Stefan Krastanov, Ryan Hamerly, Nicholas Harris, Darius Bunandar, Matthew Streshinsky, Michael Hochberg, and Dirk Englund. Single-chip photonic deep neural network with forward-only training. *Nature Photonics*, 18(12):1335–1343, 2024.

[138] Adam N McCaughan, Bakhrom G Oripov, Natesh Ganesh, Sae Woo Nam, Andrew Dienstfrey, and Sonia M Buckley. Multiplexed gradient descent: Fast online training of modern datasets on hardware neural networks without backpropagation. *APL Machine Learning*, 1(2), 2023.

[139] Ilker Oguz, Jih-Liang Hsieh, Niyazi Ulas Dinc, Uğur Teğin, Mustafa Yildirim, Carlo Gigli, Christophe Moser, and Demetri Psaltis. Programming nonlinear propagation for efficient optical learning machines. *Advanced Photonics*, 6(1):016002–016002, 2024.

[140] Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017.

[141] Su-in Yi, Jack D Kendall, R Stanley Williams, and Suhas Kumar. Activity-difference training of deep neural networks using memristor crossbars. *Nature Electronics*, 6(1):45–51, 2023.

[142] Victor Lopez-Pastor and Florian Marquardt. Self-learning machines based on hamiltonian echo backpropagation. *Physical Review X*, 13(3):031020, 2023.

[143] Clara C Wanjura and Florian Marquardt. Quantum equilibrium propagation for efficient training of quantum systems based on onsager reciprocity. *Nature Communications*, 16(1):6595, 2025.

[144] Alexander Ororbia and Ankur Mali. The predictive forward-forward algorithm. *arXiv preprint arXiv:2301.01452*, 2023.

[145] Anna Choromanska, Benjamin Cowen, Sadhana Kumaravel, Ronny Luss, Mattia Rigotti, Irina Rish, Paolo Diachille, Viatcheslav Gurev, Brian Kingsbury, Ravi Tejwani, et al. Beyond backprop: Online alternating minimization with auxiliary variables. In *Proceedings of International Conference on Machine Learning*, pages 1193–1202. PMLR, 2019.

[146] Georgios Detorakis, Travis Bartley, and Emre Neftci. Contrastive hebbian learning with random feedback weights. *Neural Networks*, 114:1–14, 2019.

[147] Wan-Duo Kurt Ma, JP Lewis, and W Bastiaan Kleijn. The HSIC bottleneck: Deep learning without back-propagation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 5085–5092, 2020.

[148] Li Ji-An and Marcus K Benna. Deep learning without weight symmetry. *ArXiv*, pages arXiv–2405, 2024.

[149] Will Xiao, Honglin Chen, Qianli Liao, and Tomaso Poggio. Biologically-plausible learning algorithms can scale to large datasets. *arXiv preprint arXiv:1811.03567*, 2018.

[150] Alexander Ororbia and Daniel Kifer. The neural coding framework for learning generative models. *Nature Communications*, 13(1):2064, 2022.

[151] Sunghyeon Woo, Jeongwoo Park, Jiwoo Hong, and Dongsuk Jeon. Activation sharing with asymmetric paths solves weight transport problem without bidirectional connection. In *Proceedings of Advances in Neural Information Processing Systems*, volume 34, pages 29697–29709, 2021.

[152] Jason Phang, Yi Mao, Pengcheng He, and Weizhu Chen. Hypertuning: Toward adapting large language models without back-propagation. In *Proceedings of International Conference on Machine Learning*, pages 27854–27875. pmlr, 2023.

[153] Mengwei Xu, Dongqi Cai, Yaozong Wu, Xiang Li, and Shangguang Wang. FwdLLM: Efficient federated finetuning of large language models with perturbed inferences. In *Proceedings of USENIX Annual Technical Conference*, pages 579–596, 2024.

[154] Archiki Prasad, Peter Hase, Xiang Zhou, and Mohit Bansal. Grips: Gradient-free, edit-based instruction search for prompting large language models. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 3845–3864, 2023.

[155] Bairu Hou, Joe O'connor, Jacob Andreas, Shiyu Chang, and Yang Zhang. Promptboosting: Black-box text classification with ten forward passes. In *Proceedings of International Conference on Machine Learning*, pages 13309–13324. PMLR, 2023.

[156] Shizhe Diao, Zhichao Huang, Ruijia Xu, Xuechun Li, Yong Lin, Xiao Zhou, and Tong Zhang. Black-box prompt learning for pre-trained language models. *arXiv preprint arXiv:2201.08531*, 2022.

[157] Xingsi Dong and Si Wu. Neural sampling in hierarchical exponential-family energy-based models. In *Proceedings of Advances in Neural Information Processing Systems*, volume 36, pages 78593–78606, 2023.

[158] Alexander G Ororbia and Ankur Mali. Backprop-free reinforcement learning with active neural generative coding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 29–37, 2022.

[159] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. LoRA: Low-rank adaptation of large language models. In *Proceedings of International Conference on Learning Representations*, volume 1, page 3, 2022.

[160] David Haussler and Manfred Warmuth. The probably approximately correct (pac) and other learning models. *The Mathematics of Generalization*, pages 17–36, 2018.

[161] Heng Xue, Zhen Li, Meilin Zhang, Hao Wang, and Rui Chen. Power-efficient spiking neural networks: Optimizing architectures, learning methods, and hardware deployment. *Authorea Preprints*, 2025.

[162] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Proceedings of Advances in Neural Information Processing Systems*, volume 36, pages 53728–53741, 2023.

[163] Dhireesha Kudithipudi, Catherine Schuman, Craig M Vineyard, Tej Pandit, Cory Merkel, Rajkumar Kubendran, James B Aimone, Garrick Orchard, Christian Mayr, Ryad Benosman, et al. Neuromorphic computing at scale. *Nature*, 637(8047):801–812, 2025.

[164] Man Yao, Jiakui Hu, Zhaokun Zhou, Li Yuan, Yonghong Tian, Bo Xu, and Guoqi Li. Spike-driven transformer. In *Proceedings of Advances in Neural Information Processing Systems*, volume 36, pages 64043–64058, 2023.

[165] Steve B Furber, Francesco Galluppi, Steve Temple, and Luis A Plana. The spinnaker project. *Proceedings of the IEEE*, 102(5):652–665, 2014.